

The Expressive Power of Graph Neural Networks: A Survey

Bingxu Zhang¹, Changjun Fan¹, Shixuan Liu¹, Kuihua Huang, Xiang Zhao², Jincai Huang², and Zhong Liu¹

(Survey Paper)

Abstract—Graph neural networks (GNNs) are effective machine learning models for many graph-related applications. Despite their empirical success, many research efforts focus on the theoretical limitations of GNNs, i.e., the GNNs expressive power. Early works in this domain mainly focus on studying the graph isomorphism recognition ability of GNNs, and recent works try to leverage the properties such as subgraph counting and connectivity learning to characterize the expressive power of GNNs, which are more practical and closer to real-world. However, no survey papers and open-source repositories comprehensively summarize and discuss models in this important direction. To fill the gap, we conduct a first survey for models for enhancing expressive power under different forms of definition. Concretely, the models are reviewed based on three categories, i.e., Graph feature enhancement, Graph topology enhancement, and GNNs architecture enhancement.

Index Terms—Approximation ability, expressive power, graph neural network, separation ability.

I. INTRODUCTION

GRAPH neural networks (GNNs) have emerged as a leading model in deep learning, attracting significant research interests [1], [2], [3] due to their superior ability in learning graph data. Their variants are widely applied in various real-world scenarios, including [4], computer vision [5], natural language processing [6], molecular analysis [7], data mining [8], financial risk control [9], [10], bioinformatics [11] and anomaly detection [12]. For more introductions to the foundations and applications of GNNs, see references [10], [13], [14], [15], [16], [17], [18] for more details.

Advancements in GNNs have garnered academic interest and hold significant potential for real-world applications, offering practical solutions to complex issues. For instance, in community detection [19], GNNs leverage mutual information

maximization to uncover structures within attribute networks. In security domain [20], sequential attention methods utilizing GNNs feature representations are adept at tracking network dynamic changes. Recommendation systems [21] also benefit from the GNNs rich nodes representation, adeptly handling extensive user behavior and item attribute data to enhance recommendation accuracy. Overall, GNNs is pivotal across various domains, enhancing task performance and pioneering novel approaches to problem-solving. This versatility cannot be achieved without the support of the powerful expressive power of GNNs, which is essential for effectively capturing and modelling the complex dynamics of real-world data.

The expressive power of neural networks (NNs) refers to their ability to represent and approximate a wide range of functions, patterns, and relationships within data. It encompasses the network's capacity to capture and model complex and intricate relationships present in the input data. NNs are known for their remarkable expressive power, which allows them to learn and represent highly non-linear and intricate relationships between input and output data. While the expressive power of GNNs refers to the scope and complexity that the model is able to express when processing graph data, specifically the model's ability to encode and process feature information on the graph as well as graph structure information. Suppose we have a social network dataset and the goal is to identify users with high influence in the network, where the authority of the user (node features) and the social scope of the user (neighborhood structure of the nodes) together determine the influence of the user. Using GNNs to capture the relationships between nodes and the topology of the network, by aggregating and transforming the node features and learning the complex patterns of the network structure, it is possible to generate node representations with rich expressive power that reflect the feature and structural relationships of the nodes, thus helping us to identify high-influence users. With this example, it can be seen that the expressive power of GNNs plays an important role when dealing with graph data by allowing us to capture the complex patterns of the network with rich representations that guide our reasoning and decision-making process. Thus, expressive power is one of the core concepts of GNNs and provides us with a powerful tool for understanding and processing graph data.

However, studying the expressive power of GNNs faces many challenges. First, Compared to well-structured texts and images, graphs are irregular. A fundamental assumption behind machine

Received 5 August 2023; revised 15 November 2024; accepted 22 December 2024. Date of publication 30 December 2024; date of current version 5 February 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 72421002, Grant 72025405, Grant 62206303, Grant U23A20296, and Grant 62272469, and in part by the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3009 and Grant 2023RC1007. Recommended for acceptance by Z. Wang. (Bingxu Zhang and Changjun Fan contributed equally to this work.) (Corresponding authors: Changjun Fan; Zhong Liu.)

The authors are with the Laboratory for Big Data and Decision, College of Systems Engineering, National University of Defense Technology, Hunan 410073, China (e-mail: zhangbingxun@nudt.edu.cn; fan-changjun@nudt.edu.cn; liushixuan@nudt.edu.cn; huangkuihua@nudt.edu.cn; zhaoxiang@nudt.edu.cn; huangjincai@nudt.edu.cn; liuzhong@nudt.edu.cn).

Digital Object Identifier 10.1109/TKDE.2024.3523700

learning on graphs is that the predicted targets should be invariant to the order of nodes on graphs. To match this assumption, GNNs introduce an inductive bias called permutation invariance [22]. Specifically, GNNs give outputs that are independent of how the node indices of the graph are assigned and the order in which they are processed, i.e., the model parameters are independent of the node order and are shared throughout the graph. Due to this new parameter sharing mechanism, we need new theoretical tools to study their expressive power. Moreover, most GNNs are usually used as black-box feature extractors for graphs, and it is unclear how well they can capture different graph features and topology. Second, due to the introduction of permutation invariance, the results of the classical universal approximation theorem of NNs [23], [24] cannot be directly generalized to GNNs [25], [26], [27]. In addition, in practice, the study of expressive power is related to a number of long-standing difficult problems in graph theory [28], [29]. For example, in the prediction of properties of chemical molecules, it is necessary to judge whether the molecular structure is the same or similar to that of a molecule with known properties, which involves the problem of graph/subgraph isomorphism judgement [28], [30] and graph matching [31], [32], [33], etc.

There exists a pioneering study of the expressive power of GNNs. Morris et al. [34], [35] and Xu et al. [36] introduced to use graph isomorphism recognition to explain the expressive power of GNNs, leading the trend to analyze the separation ability of GNNs. Maron et al. [25] and Chen et al. [37] introduced to use the ability of GNNs to approximate graph functions to account for their expressive power and further gave the set representation of invariant graph functions that can be approximated by GNNs, leading the trend to analyze the approximation ability of GNNs. While multiple studies have emerged in recent years that characterize and enhance the expressive power of GNNs, a comprehensive review in this direction is still lacking. Sato [38] explored the relationship between graph isomorphism testing and expressive power and summarized strategies to overcome the limitations of GNNs expressive power. However, they only describe the GNNs separation ability and approximation ability to characterize the GNNs expressive power, and there also exists other abilities, including subgraph counting ability, spectral decomposition ability [39], [40], [41], logical ability [42], [43], [44], [45], [46], [47], [48], etc., which are also considered as the main categories of GNNs expressive power. Therefore, it is crucial to clarify the definition and scope of the GNNs expressive power, which motivates this work.

In this work, we explore GNNs expressive power through two critical components: feature embedding and topology representation. The latter capability distinguishes GNNs within other machine learning models. Our analysis reveals that both features and topology influence GNNs expressive power, with topology learning and keeping deficiencies being a major limitation.

This paper categorizes methods to enhance GNNs expressive power into three main strategies: graph feature enhancement, which boosts embedding effects; graph topology enhancement, aimed at more effectively representing complex topological information; and GNNs architecture enhancement, focusing on refining aggregation functions that restrict expressivity. We

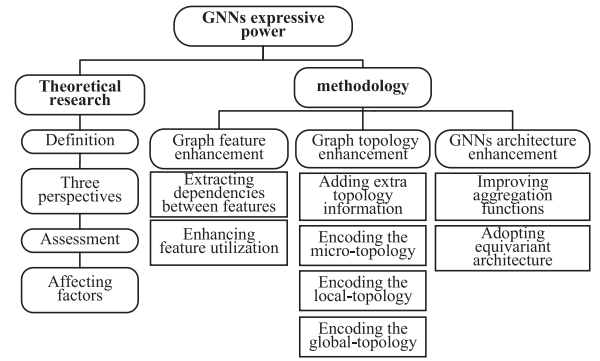


Fig. 1. The specific logical structure of the article.

TABLE I
NOTATIONS IN THE SURVEY

Notations	Descriptions
$\{\}$	A set.
$\{\{\}\}$	A multiset.
$G = (\mathcal{V}, \mathcal{E})$	An unfeatured graph.
$\mathcal{V}$	The set of nodes in a graph.
$\mathcal{E}$	The set of edges in a graph.
$N(v)$	The set of the neighboring nodes of node v .
$G = (\mathbf{A}, \mathbf{X})$	An featured graph.
$\mathbf{A}$	The adjacency matrix.
$\mathbf{X}$	The feature matrix.
$\mathbf{h}_v^{(l)}$	The embedding of node v in the l -th layer.
$\mathbf{H}^{(l)}$	The embedding matrix in the l -th layer.
$\text{MSG}^{(l)}$	The message passing function in the l -th layer.
$\text{AGG}^{(l)}$	The aggregation function in the l -th layer.
$\text{UPD}^{(l)}$	The update function in the l -th layer.

spotlight the deficiencies in current benchmarks and metrics for GNNs expressive power, underscoring the obstacles in gauging it. We also propose innovative research trajectories, such as physics-informed GNN design and the exploration of alternative graph transformer models. This work is organized as follows: Section II lays out foundational knowledge on GNNs and graph isomorphism. Section III delves into the importance of GNNs expressive power, offering a unified definition, research angles, evaluation metrics, and affecting factors. Section IV presents strategies to improve the expressive power, overviewing the high-expressivity GNNs with their pros and cons. Section V discusses the challenges and opportunities in research. Finally, we conclude with Section VI, with the article's logical flow depicted in Fig. 1.

II. PRELIMINARY

Before presenting the relevant definitions and the problem description, Table I gives a summary of all the notations used in the article.

A. Basics of Graph Neural Networks

Graph: In the following, we consider the problem of representation learning on one or several graphs of possibly varying

sizes. Let $G = (\mathcal{V}, \mathcal{E}) \in \mathcal{G}$ be a graph of n nodes with node set $\mathcal{V}$ and edge set $\mathcal{E}$, directed or undirected. Each graph has an adjacency matrix $\mathbf{A} \in \{0, 1\}^{n \times n}$, and potential node features $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_n)^T \in \mathbb{R}^{n \times c}$. Combining the adjacency matrix and node features, we may also denote $G = (\mathbf{A}, \mathbf{X})$. When explicit node features are unavailable, an unfeatured graph G will consider $\mathbf{X} = \mathbf{1}\mathbf{1}^T$ where $\mathbf{1}$ is a $n \times 1$ vector of ones. For a node $v \in \mathcal{V}$, the set of neighboring nodes is represented as $N(v)$. We also use $\mathcal{V}[G]$ to denote the node set of graph G .

Message Passing Neural Networks (MPNNs): Out of the numerous architectures proposed, MPNNs have been widely adopted [22], [49], [50]. MPNNs employ a neighborhood aggregation strategy to encode each node $v \in \mathcal{V}$ with a vector representation $\mathbf{h}_v$. This node representation is updated iteratively by gathering representations of its neighbors before performing a nonlinear transformation of them with neural network (NN) layers:

$$\text{MessagePassing} : \mathbf{m}_u^{(l)} = \text{MSG}^{(l)} \left(\left\{ \mathbf{h}_u^{(l-1)} : u \in N(v) \right\} \right), \quad (1)$$

$$\text{Aggregation} : \mathbf{a}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \mathbf{m}_u^{(l-1)} : u \in N(v) \right\} \right), \quad (2)$$

$$\text{Update} : \mathbf{h}_v^{(l)} = \text{UPD}^{(l)} \left(\mathbf{h}_v^{(l-1)}, \mathbf{a}_v^{(l)} \right). \quad (3)$$

where, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, $\mathbf{m}_u^{(l)}$ represents the message from the neighbors, $\mathbf{m}_v^{(l)}$ represents the aggregated message from neighbors at layer l , $\text{MSG}^{(l)}$, $\text{AGG}^{(l)}$ and $\text{UPD}^{(l)}$ denote the message passing, aggregation and update functions of layer l .

The neighbor aggregation strategy employed by MPNNs allows them to learn relationships between nearby nodes. This inductive bias is well suited for problems requiring relational reasoning [51], [52], [53], [54], [55]. Despite their widespread success, MPNNs nonetheless suffer from several limitations, including over-smoothing [56], [57], [58], over-compression [59], [60], the failure to distinguish between node identity and location [61], [62], and limited expressive power [34], [36]. The lack of expressive power is evident in their inability to compute several crucial graph properties (e.g., shortest/longest cycles, diameters and local clustering coefficients [63]), and their failure to learn the graph topology (for example, whether a graph is connected or whether a particular pattern, such as a cycle, exists in the graph [64]). To address tasks where the graph topology is critical, such as predicting the chemical properties of molecules [65], [66] and solving combinatorial optimization problems [67], [68], more powerful GNNs are required.

k -order Linear Invariant GNN (k -IGN) Framework: The k -IGN by Maron is another widely used framework for investigating GNN's expressive power [25], [69]. The k -IGN comprises of alternating layers composed of invariant or equivariant linear layers and nonlinear activation layers. Each linear function is given as L , and the non-linear activation function is denoted as σ . The parameter k in k -IGN refers to the order of the tensor $\mathbf{A}$, which corresponds to the indices used to represent its elements. For instance, a first-order tensor represents the node feature, a

second-order tensor represents the edge feature, and a k -th order tensor encodes hyper-edge feature. For $k = 2$, the linear layer consists of 15 equivariant layers and 2 invariant layers, which can be expressed as follows:

$$L(\mathbf{A}) = \sum_{i=1}^{15} \theta_i L_i(\mathbf{A}) + \sum_{i=16}^{17} \theta_i \bar{L}_i(\mathbf{A}) \quad (4)$$

After adding the non-linear activation layer, the 2-IGN architecture can be presented as:

$$\mathbf{A}^{(l+1)} = \sigma \left(L^{(l+1)} \left(\mathbf{A}^{(l)} \right) \right) \quad (5)$$

IGN has demonstrated not less expressive power than MPNN and is therefore being widely adopted to augment expressiveness. Aside from the k -IGN frameworks, there also have been efforts to construct more powerful equivariant networks by combining a set of simple permutation equivariant operators [44], [45], [46], [47], [48], [70].

Permutation Invariance: Graphs are inherently disordered, with no fixed node order. As a basic inductive bias in graph representation learning, permutation invariance refers to the fact that the output of a model is independent of the input order of nodes. Specifically, a model f is permutation invariant if it satisfies $f \circ \pi(G) = f(G)$ for all possible permutations π .

Permutation Equivariance: On the other hand, permutation equivariance means that the output order of the model corresponds to the input order. When we focus on node features, we expect the model to output in the same order as the input nodes. Specifically, a model g is permutation invariant equivariant if it satisfies $g \circ \pi(G) = \pi \circ g(G)$ for all possible permutations π .

The design philosophy of GNNs is to generate a permutation equivariant function on the graph by applying a local permutation invariant function to aggregate the neighbor features of each node [69].

B. Basics of Graph Isomorphism

Graph Isomorphism: Graph isomorphism refers to the topological equivalence of two graphs. Given two graphs $G = (\mathbf{A}, \mathbf{X})$ and $G' = (\mathbf{A}', \mathbf{X}')$ with adjacency matrices $\mathbf{A}$ and $\mathbf{A}'$ and feature matrices $\mathbf{X}$ and $\mathbf{X}'$ respectively, an isomorphism between these graphs is a bijective mapping $\pi : \mathcal{V}[G] \rightarrow \mathcal{V}[G']$ that satisfies $\mathbf{A}_{uv} = \mathbf{A}'_{\pi(u)\pi(v)}$ and $\mathbf{X}_v = \mathbf{X}'_{\pi(v)}$. We use $G \cong G'$ to denote that G and G' are isomorphic, where G' can be represented as $\pi(G)$. If there is no bijection π that satisfies these conditions, we say that G and G' are non-isomorphic.

Weisfeiler-Lehman test (WL test): Graph isomorphism problems are notoriously difficult and are classified as NP-hard in graph theory. The WL test of isomorphism [71], [72], also known as the color refinement algorithm [73]), is an effective algorithm to solve graph isomorphism problems. The WL test utilizes a graph coloring technique to assign color labels to each node in the graph. This iterative process is illustrated in Fig. 2 [74]. Initially, every node in the graph is assigned a unique color label c . In each iteration, the central node and its neighbors are combined to form a set of multiple colors, which are subsequently mapped to a new unique color using

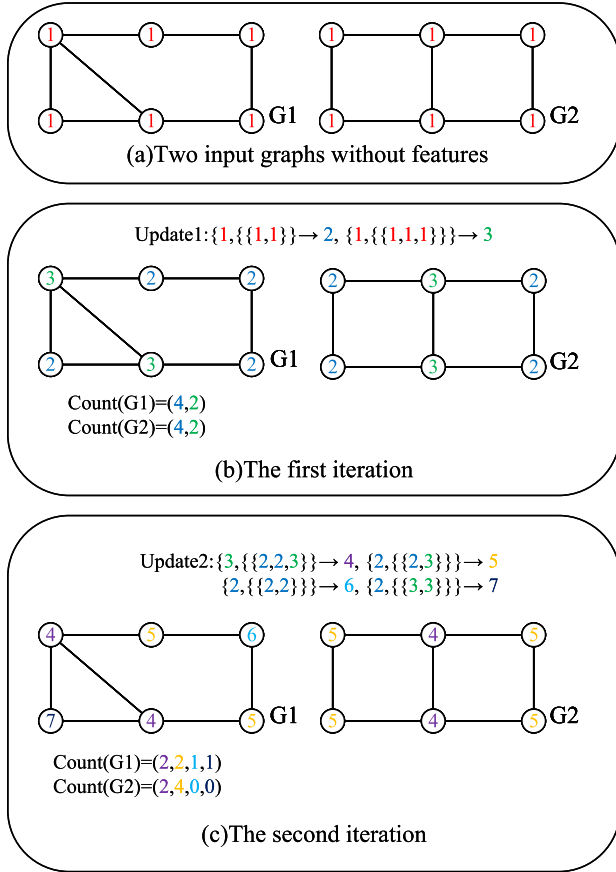


Fig. 2. The illustration of the aggregation and update process of WL test. (a) Given two graphs without features, and add color labels to all nodes. (b) In the first iteration, the different information aggregated by the nodes is mapped into new color labels, and then these new labels are redistributed to nodes, and the number of labels is counted after the distribution. After 1st iteration, G1 and G2 have the same color distributions to determine whether they are isomorphic, and the next iteration is performed. (c) Perform the node neighbor aggregation and redistribution of color labels steps again, and get the different color distributions of G1 and G2, at this time, it can be determined that the two are non-isomorphic.

Hash function, which becomes the new label of the central node. The process is repeated until the node color labels no longer change, resulting in a color distribution histogram of the graph. By comparing the color distributions of two graphs, the WL test can determine whether they are isomorphic or not. This process can be divided into several steps, neighbor label aggregation, multi-sets sorting, label compression, and label update and can be mathematically formulated as follows:

$$\mathbf{c}_v^{(l)} = \text{Hash} \left(\mathbf{c}_v^{(l-1)}, \left\{ \mathbf{c}_u^{(l-1)} \mid u \in N(v) \right\} \right). \quad (6)$$

It is important to note that identical coloring results could only suggest but cannot guarantee graph isomorphism.

While the WL test is fast and efficient for most graphs [75], it has limitations in distinguishing members of many important graphs, such as regular graphs. The more powerful k-WL algorithm was initially introduced by Weisfeiler [76], expanding the coloring of individual vertices to the coloring of k-tuples. A k-tuple is a unit of k neighbored nodes, and the k-WL test [26], [27] captures higher-order interactions in the graph

by considering the process of iterative aggregation between all neighbored k-tuple units. Specifically, the labels for tuple units consisting of k nodes are computed, treating these tuple units as significant graph entities for iterative operations. The k-WL becomes increasingly expressive as k grows. For sufficiently large k, it can distinguish any finite set of graphs. It has been observed that the discrimination potential for $k > 1$, (k+1)-WL test is unequivocally stronger than the k-WL test, creating a hierarchical structure where the discriminative power increases with higher k values. Therefore, the k-WL test excels at detecting subgraph topology with larger k values, exhibiting an enhanced discrimination ability concerning graph structures.

K-WL iteratively recolors all n^k k-tuples defined on a graph G with n nodes. In iteration 0, each k-tuple $\vec{v} = (v_1, \dots, v_k) \in V(G)^k$ is initially assigned a color as its atomic type $c_k^{(0)}(\vec{v})$. If G has d colors, then $c_k^{(0)}(\vec{v})$ is an ordered vector encoding. Let $c_k^{(l)}(\vec{v})$ denote the color of k-tuple $\vec{v}$ on graph G at the l -th iteration of the k-WL. In the l -th iteration, k-WL updates the color of each $\vec{v}$ in $V(G)^k$ according to:

$$\begin{aligned} c_k^{(l+1)}(\vec{v}) &= \text{Hash}(c_k^{(l)}(\vec{v}), \\ &\quad \{ \{ c_k^{(l)}(\vec{v}[x/1] \mid x \in V(G) \} \}, \dots, \\ &\quad \{ \{ c_k^{(l)}(\vec{v}[x/k] \mid x \in V(G) \} \} \}). \end{aligned} \quad (7)$$

Let $c_k^{(l)}(G)$ denote the encoding of G at l -th iteration of k-WL. Then,

$$c_k^{(l)}(G) = \text{Hash}(\{ \{ c_k^{(l)}(\vec{v}) \mid \vec{v} \in V(G)^k \} \}). \quad (8)$$

Moreover, k-WL has interesting connections to logic, games, and linear equations [43], [46], [77]. Another variation of k-WL is known as k-folklore WL (k-FWL), where it is shown that the expressive power of (k+1)-WL is equivalent to that of k-FWL [74].

III. THE DESCRIPTION OF GNNs EXPRESSIVE POWER

A. The Necessity of GNNs Expressive Power

Expressive power: All machine learning problems can be abstracted as learning a mapping f^* from the feature space $\mathcal{X}$ to the target space $\mathcal{Y}$. f^* is usually approximated using a model f_θ by optimizing some parameters θ . In practice, f^* is typically a priori unknown, so one would like f_θ to approximate a wide range of f^* as much as possible. An estimate of how wide this range is called the *expressive power* of the model, providing an important measure of the model potential [78], shown in Fig. 3(a).

The powerful expressive power of NNs is reflected in their ability to approximate all continuous functions [79], specifically the ability to embed data within the feature space $\mathcal{X}$ into the target space $\mathcal{Y}$ generated by any continuous function, which is in fact feature embedding ability, show in Fig. 3(b). Due to the strong expressive power of NNs, similarly, few works [80], [81], [82], [83] doubt the expressive power of GNNs that have demonstrated significantly superior performance in a variety of application tasks because they naturally feature the superior

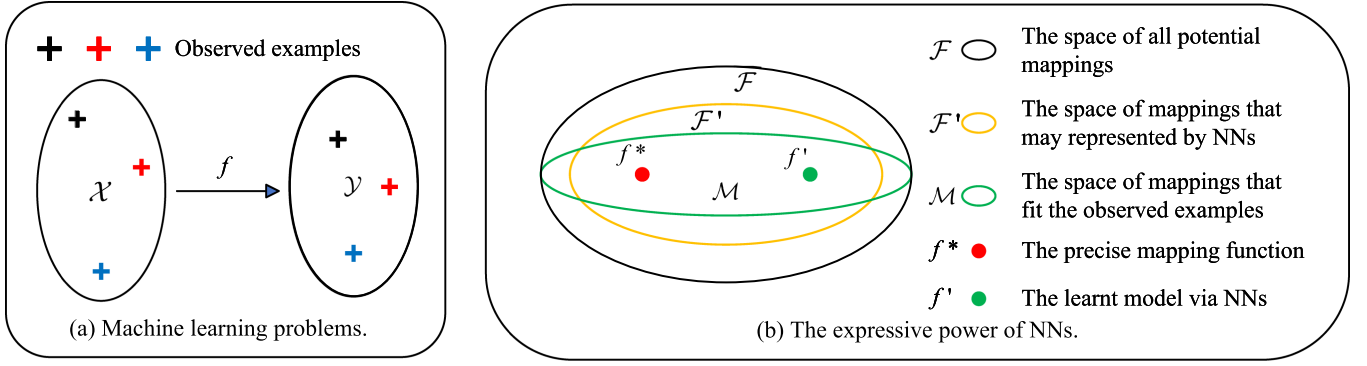


Fig. 3. An illustration of the expressive power of NNs and their effects on the performance of learned models. (a) Machine learning problems aim to learn the mapping from the feature space to the target space based on several observed examples. (b) The expressive power of NNs refers to the gap between the two spaces $\mathcal{F}$ and $\mathcal{F}'$. Although NNs are expressive ($\mathcal{F}'$ is dense in $\mathcal{F}$), the learned model f' based on NNs may differ significantly from f^* due to their overfit of the limited observed data. This figure is with reference to Figure. 5.5 in [78].

performance of GNNs to their excellent feature embedding ability.

However, some graph-augmented multilayer perceptron (MLP) models [84], [85] outperform GNNs despite the former's less expressive power than the latter in a number of node classification problems [86], where MLPs [87] use only information from each node to compute feature embeddings of nodes, and GNNs iteratively aggregate the features of neighbouring nodes in each layer, thus allowing the use of global information to compute feature embeddings of nodes. This fact results in a contradiction between our intuitive understanding of expressive power, yet at the same time illustrates that feature embedding power does not describe the expressive power of GNNs well.

GNNs, compared to NNs, add the inductive bias of permutation invariance so that they can propagate and aggregate information over the topology of the graph. [57], [88] demonstrate that if GNNs only have feature embedding ability but lack the ability to maintain the graph topology, then their performance may be inferior to MLP. In addition, we know that the expressive power of feed-forward neural networks is often limited by their width, whereas the GNNs' expressive power is not only limited by their width, but also how they use the graph topology for message propagation and node updates. It follows that the topology representation ability of GNNs, i.e., the ability to maintain the graph topology, becomes the key to their superior performance. To characterise the expressive power of GNNs using their topology representation ability then requires a new set of theoretical tools.

B. The Definition of GNNs Expressive Power

Graph data typically contains both feature and topology information. Therefore, the expressive power of GNNs refers to the ability of the model to encode and process feature information on the graph as well as the structural information of the graph. A GNN model with high expressive power can not only capture the complex relationships in the feature, but also the patterns of the graph structure, and then transform them into meaningful representations. The level of expressive power directly affects the performance and effectiveness of the

model. Here, the feature embedding ability represents the part of GNNs model that encodes and processes feature information, and the topology representation ability embodies the model's ability to encode and process structural information. As a neural network model for graph representation learning in the family of NNs, the feature embedding ability of GNNs is unquestionable, and their expressive power is more reflected in their topology representation ability. GNNs map node v from original space $\mathcal{X}$ to embedding space $\mathcal{Y}$ to obtain node embedding $\mathbf{h}_v$ via function f , where $\mathbf{h}_v = f(\mathbf{A} \cdot \mathbf{x}_v)$. For nodes v_1, v_2 in different topological positions on graph G , if $\mathbf{x}_1 \neq \mathbf{x}_2$, the results of the feature embedding ability ensure that $f(\mathbf{A} \cdot \mathbf{x}_1) \neq f(\mathbf{A} \cdot \mathbf{x}_2)$. And if $\mathbf{x}_1 = \mathbf{x}_2$, to maintain the topology of the graph, it meanwhile requires that $f(\mathbf{A} \cdot \mathbf{x}_1) \neq f(\mathbf{A} \cdot \mathbf{x}_2)$. The relevant formal definitions are further given below.

Feature embedding ability: Let $\mathbf{X} = \mathbf{random}$, i.e., input graph with node features, the feature embedding ability can be expressed by the size of the value domain space $\mathcal{F} = \{f(\mathbf{A} \cdot \mathbf{X}) | \mathbf{X} = \mathbf{random}\}$ of f .

Topology representation ability: Let $\mathbf{X} = \mathbf{1}$, i.e., input graph without node features, the topology representation ability can be expressed by the size of the value domain space $\mathcal{F}' = \{f(\mathbf{A} \cdot \mathbf{X}) | \mathbf{X} = \mathbf{1}\}$ of f .

The feature embedding ability reflects that nodes with different features can get different node embeddings. Topology representation ability reflects that nodes with different topological positions can get different node embeddings. Together, these two constitute the expressive power of GNNs, illustrated with reference to Fig. 4.

The expressive power of GNNs: Let $\mathbf{X} = \mathbf{random}$, the size of the space $\mathcal{F} \cap \mathcal{F}'$ describes the expressive power of f . Typically, $\mathcal{F} \supseteq \mathcal{F}'$, which means that the topology representation ability is the main factor limiting the expression of GNNs. And for two models, f_1 and f_2 , if $\mathcal{F}_1 \supset \mathcal{F}_2$, we say that f_1 is more expressive than f_2 .

C. The Existing Perspectives of GNNs Expressive Power

Several prevailing views in current research on the expressive power of GNNs characterise the expressive power as

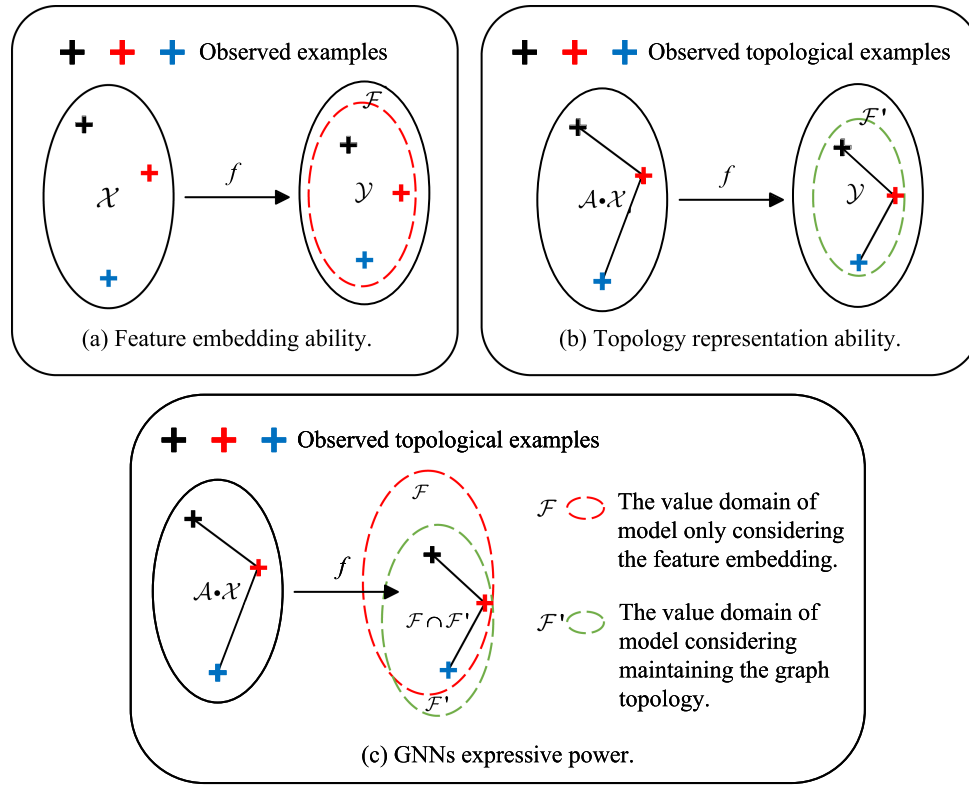


Fig. 4. An illustration of the expressive power of GNNs. (a) The feature embedding ability of GNNs is the same as that of NNs in that both maps the observed examples in the feature space $\mathcal{X}$ to the target space $\mathcal{Y}$ via f . The strength of the feature embedding ability is measured by the size of the value domain space $\mathcal{F}$ of f . (b) The topology representation capability of GNNs is achieved by mapping the observed examples in the feature space to the target space via f and preserving the original topology between the examples. The strength of the capability is measured by the size of the value domain space $\mathcal{F}'$ with $\mathbf{X} = 1$. (c) The expressive power of GNNs consists of a combination of feature embedding ability and topology representation ability, measured by the size of the intersection of $\mathcal{F}$ and $\mathcal{F}'$ with $\mathbf{X} = \text{random}$.

approximation ability, separation ability and subgraph counting ability, respectively.

Approximation ability: This perspective follows the path of classical neural network expressive power research, which uses the ability of GNNs to approximate functions on graphs to characterize expressive power, and the focus of the research is actually on examining the feature embedding ability of the model. In traffic network flow prediction, GNNs update the representation of each road node by aggregating the flow information of neighboring road sections and considering time series features. In this way, GNNs learn a function capable of mapping current and past traffic states to future flows. A GNN model with high approximation capability is able to accurately predict future flows because it is able to effectively understand and model complex traffic flow patterns and interactions between road networks. The higher approximation ability ensures that GNNs are able to process graph data more efficiently while maintaining a high level of accuracy. However, further exploration of the types of functions that GNNs can approximate is still insufficient to solve more and more complex practical problems.

Separation ability: This perspective builds on the framework of GNNs expressive power research based on the WL test, which uses the ability of GNNs to recognize non-isomorphic graphs to describe expressive power, and the focus of the research is actually on examining the topology representation ability of

the model. This ability is important when dealing with large graphs with complex structures. For example, in social networks, people may have multiple different social circles, and GNNs need to be able to recognize these circles and separate them. However, the scope of research on specific separation ability is limited to featureless graphs, which leads to their lack of practical applicability in solving real-world problems.

Subgraph counting ability: This perspective also lies in examining the topology representation ability of models, using the ability of GNNs to recognize and count substructures on graphs to describe the expressive power. Unlike separation ability, which focuses on structural differences between pairs of graphs, subgraph counting ability focuses more on distinguishing subgraph structures within a single graph. Unlike the whole graph structures, subgraph structures are highly relevant to molecular functional groups and communities in real-world tasks, such as predicting the properties of a molecule in chemistry, which requires counting different structures in a molecule [49], and thus have a higher value for practical applications. In predicting specific properties of a molecule, GNNs identify specific substructures in the molecule, such as benzene rings or hydroxyl groups, by learning the topology of the graph. Then, by counting the number of these substructures in the molecule, GNNs can capture key information that is closely related to the properties of the molecule. For example, the number of benzene rings may

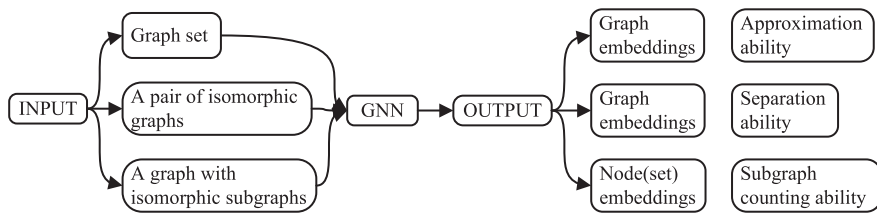


Fig. 5. Input and output of the GNNs model under different expressive power representations. When approximate ability is used to characterise the expressive power, the input of the model is a set of graphs and the output is a graph embeddings. When separation ability, the input is a pair of graphs and the output is graph embeddings. When the subgraph counting ability, the input is a single graph and the output is a node (set) embeddings.

affect the stability and reactivity of the molecule; the presence and number of hydroxyl groups may determine the solubility of the molecule.

1) Approximation Ability: Models with stronger expressive power can learn more complex mapping functions. Neural networks are known for their strong expressive power, and one of the most famous results is the universal approximation theorem for neural networks [23], [24], [79]. As a member of the neural network family, GNNs can also use the ability to approximate mapping functions to describe their own expressive power. However, because GNNs add inductive bias [22] to graph learning, i.e., permutation invariance, the results of the generalized approximation theorem for neural networks cannot be simply generalized to GNNs. In this context, Maron et al. [25], [26], [27] describe specifically the types of functions that GNNs can approximate, using the *approximate ability* to approximate to describe the expressive power of GNNs, that is, what kind of permutation invariant graph functions GNNs can approximate. They further give a description of the set of functions that the model can approximate as a measure of expressive power. Assume that GNN is a mapping $f_\theta : \mathcal{G} \rightarrow R^{d \times n \times m}$ on graph, where $\mathcal{G}$ is the set of graphs $G = (\mathbf{A}, \mathbf{X})$, $\mathbf{X} \in R^{d \times n}$ is node feature, d is the dimension of the feature, $n = |V|$ and $m = |G|$. Denote the output of GNNs by $f_\theta(G)$, the formal definition of approximate ability can be given.

Function approximation: Consider a function f^* to be learned, when the input of GNNs is the set of graphs $\mathcal{G}$, if $\|f^*(G) - f_\theta(G)\|_\infty := \sup_G |f^*(G) - f_\theta(G)| < \varepsilon$, for $G \in \mathcal{G}$ and $\varepsilon > 0$, then f^* is successfully approximated by f_θ .

Approximation ability: The measure of the size of the set of functions composed of all potential f_θ is the approximation ability of GNNs. Fig. 5 shows the representation of input and output under the definition of approximation ability.

However, these theoretical findings are not sufficient to explain the unprecedented success of GNNs in practice, because simply knowing what graph functions the model can approximate does not guide practical production work. It is challenging to analyze the expressive power of GNNs starting from a practical application perspective.

2) Separation Ability: Considering how GNNs make predictions about molecules, GNNs should be able to predict whether a topology corresponds to a valid molecule by identifying whether the topology of the graph is the same or similar to the topology corresponding to a consistent valid molecular. This problem of

measuring whether two graphs have the same topology involves the problem of graph isomorphism [28], [30].

In the graph isomorphism perspective, Xu et al. [36] and Morris et al. [34] used *separation ability* to describe the expressive power of GNNs, that is, whether GNNs can effectively identify non-isomorphic graph topology. By gaining insight into the high similarity and close connection between the WL test and GNNs in the iterative aggregation process, they further used the WL test as a measure of the strength of the separation ability and indicated that the upper limit of the separation ability is 1-WL test. Similarly, the formal definition of the separation ability can be given.

Graph isomorphism recognition: Consider two non-isomorphic graphs $G_1 = (\mathbf{A}_1, \mathbf{X}_1)$ and $G_2 = (\mathbf{A}_2, \mathbf{X}_2)$, where $X_1 = X_2 = 1$. If $f_\theta(G_1) \neq f_\theta(G_2)$, then it is assumed that GNNs can identify non-isomorphic graph pairs.

Separation ability. The measure of the size of the set of all identifiable possible non-isomorphic pairs $|\mathcal{G}_1|$ is the separation ability of GNNs. Fig. 5 shows the representation of input and output under the definition of separation ability.

The results of the separation ability show the role that GNNs expressive power plays in practice. And in fact, it has been shown that both separation and approximation ability are theoretically unified. Chen et al. [37] demonstrated the equivalence between the two abilities, showing that the basis of the approximation ability is actually the embedding of two non-isomorphic graphs into different points of the target space, and that a model that can distinguish all nonidentical graphs is also a general function approximator. The result of equivalence provides, in a sense, a practical application for approximation ability.

However, the result of separation ability requires comparison of pairs of graphs, which requires a prior a different GNN for each pair of graphs to distinguish them. This is of little help in practical learning scenarios, because people prefer to give meaningful representations to all graphs through a single GNN. In addition, the graph isomorphism problem measures the expressive power at the graph level, while the expressive power at the node level is equally important, and whether this theory can explain the expressive power of nodes is still questionable [89].

3) Subgraph Counting Ability: Considering solving problems at the node level is also very instructive for practice. Continuing with the prediction problem of predicting molecular functions, in addition to the need to identify whether the graph topology is the same as the known functional group topology to determine the molecular properties, sometimes the number

of functional groups also has an impact on the function of the molecule, which requires determining how many identical graph topology there are in the graph. To address this issue, Chen et al. [64] have proposed to use *subgraph counting ability* to describe the GNNs expressive power, emphasizing the GNNs' ability to recognize subgraph topology in graphs. On the one hand, this ability can evaluate node-level embeddings, comparable to the separation ability. On the other hand, real-world graph topology such as molecular and community topology typically involve subgraph topology, Hence this ability has more practical guidance for applications compared to the approximation ability. Similarly, the formal definition of the separation ability can be given.

Subgraph counting: Consider two isomorphic subgraphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ of graph $G = (V, E)$, where $V_1, V_2 \in V$, $E_1, E_2 \in E$ and $V_1 \neq V_2$. If $f_\theta(G_1) \neq f_\theta(G_2)$, then it is assumed that GNNs can count subgraphs.

Subgraph counting ability: The measure of the size of the set of all identifiable countable isomorphic subgraphs $|\mathcal{G}_1|$ is the subgraph counting ability of GNNs. Fig. 5 shows the representation of input and output under the definition of subgraph counting ability.

D. The Assessment of GNNs Expressive Power

The current assessment of the strength of the expressive power of GNNs focuses more on assessing the strength of the topology representation of the model. According to the results of separation ability, the upper limit of the expression power of GNNs is the 1-WL test, the conclusion that comes from the observations of Xu et al. [36] and Morris et al. [34], who were the first to gain insights into the high degree of similarity between the neighborhood aggregation operation of GNNs and the aggregation of neighborhood labels in the WL test, and to use the WL test to assess the strength of the expressive power. However, as more and more models beyond the upper limit of the 1-WL test's ability were developed, the results of higher-order WL tests were used to assess the expressive power of these models, and the upper limit of expressive power was extended to the 3-WL test. However, the results of the k-WL test can only provide a simple general classification of the model's expressive power, and descriptive evaluation results such as "equivalent to the 1-WL test" and "not exceeding the 3-WL test" are useless in guiding the development and design of models and their practical applications scientifically and accurately. To address this issue, Puny et al. [90] introduced polynomial expressivity, utilizing graph polynomials to measure the expressive power of GNNs. This method quantifies both node and edge-level expressive power by considering polynomial functions on the adjacency matrix. Although it offers a detailed evaluation, its reliance on complex mathematics may limit its capability in fully capturing all graph structural features. Building on this concept, Zhang et al. [91] proposed homomorphic expressivity. This technique evaluates a model's capacity to encode substructures under homomorphic counting, providing a comprehensive assessment tool for comparing the expressive power of different GNN models directly. While the WL hierarchy, polynomial expressivity,

and homomorphic expressivity offer insights into assessing the expressive power, they primarily concentrate on graph structure, neglecting the importance of feature information. Recognizing that topological representation alone is insufficient, the role of feature embedding in enhancing model performance is also crucial. Interaction between features can effectively improve model performance [92]. Furthermore, GNNs topology representation ability is also significantly improved after considering into the feature embedding ability. A compelling demonstration by Kanasoulis et al. [41] showed the ability of gnn to recognize any graph with at least one node displaying unique features. This finding reaffirms our reasonable definition of the expressive power of GNNs, emphasizing that feature embedding supports the topology representation of GNNs. In particular, if the node features adequately describe the node constants, Loukas et al. [93], [94], [95] confirmed that GNNs with strong message passing and update functions become Turing universal. That is, if the feature information is sufficient for us to distinguish between nodes, the structural information plays little role in the process of node differentiation. This is confirmed in a small example of the graph isomorphism problem [96].

Strong topology representation ability and strong feature embedding ability determine the power of GNNs models to be efficiently expressed, allowing them to perform well in downstream tasks. Currently, the research community has not yet developed an evaluation method that can simultaneously and comprehensively assess the ability of GNNs to express both graph structure and feature information, and the comprehensive expressive power of GNNs is usually only indirectly measured through experimental evaluation.

E. The Factors Affecting GNNs Expressive Power

The expressive power of GNNs is influenced by both the chosen graph information encoding method and the design of the model aggregation function. The selection of the graph information encoding method, being a more overarching factor, impacts the representation of all GNN models. Conversely, the design of the model aggregation function is more focused and tailored, affecting the expressiveness of specific models within the GNN framework.

The way of encoding graph information affects expressive power by influencing the representation of graph features and graph topology. First, node feature information has strong discriminative power. GNNs can distinguish between graphs with different node features [93] and any graph that differs in at least one node feature [41]. Second, information about relationships between features also has recognition ability, and GNNs can distinguish graphs with different global graph features [47]. In addition, structural information is also recognizable, and GNNs can distinguish a pair of non-isomorphic graphs [55], as well as recognize different subgraph structures on a single graph [64]. The classical GNNs model only considers the coded node feature information, and the expressive power of the model can be further improved by further considering the relationship information between coded features and the graph structure information.

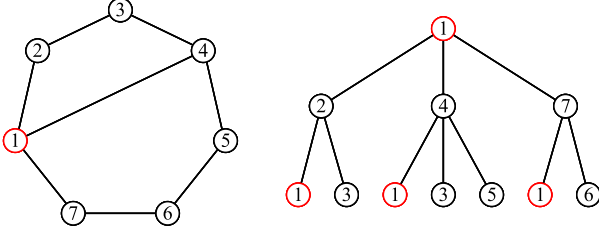


Fig. 6. The rooted subtree in message passing. The left one represents the original graph G , and the objective is to obtain an embedding of node v_1 on G , where node v_1 is denoted by red color. The right one is the subtree obtained for node v_1 after two rounds of iteration.

The aggregation function also affects the representation of graph features and graph topology. First, GNNs do not directly encode the graph topology for learning, but use the topology of the graph for information transmission and aggregation, which is realized by the permutation invariant aggregation function. This design hinders the expression of GNNs [97]. Because the permutation invariant aggregation function assumes that the states of all adjacent nodes are equal, the relationship between adjacent nodes is ignored. The node encoding obtained through neighborhood aggregation represents the rooted subtree around the central node, as illustrated in the Fig. 6. However, rooted subtrees have limited expressive power for representing non-tree graphs. Therefore, the central node cannot distinguish whether two adjacent nodes are adjacent or not, and cannot identify and reconstruct the graph topology. General GNNs can only explicitly reconstruct a star graph from a 1-hop neighborhood, but cannot model any connections between neighborhoods [64]. Therefore, the permutation invariant aggregation functions cause GNNs to lose the topology of the context in the topology representation and thus fail to learn the basic topological properties of the graph [63]. Second, under the condition that the permutation invariance of the aggregation function is guaranteed, the specific aggregation affects the model's handling of the node features, e.g., the maximum and minimum aggregation functions lead to a lower utilization of the node feature information in the model, while the mean value as well as other aggregation functions such as the attention mechanism increase the utilization of the node feature information and thus improve the model's representation. The classical GNNs model needs to maintain the permutation invariance of the aggregation function, based on which designing more efficient aggregation functions that utilize the feature information can improve the expressive ability of the model. In addition, considering other more effective means to maintain the permutation invariance of the graph instead of using the aggregation function can also further improve the expressive ability of the model.

IV. EXISTING WORK OF IMPROVING GNNs EXPRESSIVE POWER

A. Methods to Improve GNNs Expressive Power

Improving GNNs expressive power involves bolstering feature embedding and topology representation. Feature

embedding enhancement involves extracting feature dependencies and adding related features. Topology representation can be improved by encoding topology directly or optimizing GNNs architecture to overcome limitations from permutation invariant aggregation functions. The strategies can be categorized into three methods:

Graph feature enhancement aims to enrich feature embeddings by *extracting dependencies between features* to obtain more informative and expressive embedding results, or by *enhancing feature utilization* to obtain a comprehensive feature representation.

Graph topology enhancement aims to encode graph topology by *adding extra topology information* manually to improve nodes' distinctiveness or directly encoding the graph topological structure. The latter can be classified into *encoding micro-topology* (capturing distance or position information between nodes to enhance the perception of graph structure), *encoding local-topology* (using subgraphs or higher-order connectivity patterns to identify complex topological patterns), and *encoding global-topology* (directly capturing global structural attributes such as connectivity and clustering structure).

GNNs architecture enhancement aims to address limitations in learning graph topology due to permutation invariant aggregation functions. This can be achieved by *improving aggregation functions* to better capture topological information or by *adopting equivariant architectures* to overcome permutation invariance limitations, enabling the computation of global features.

Consistent with expectations, the three aforementioned methodologies encompass all currently known model designs capable of improving expressive power. Table II examines and systematically categorizes the design of more powerful expressive GNNs in recent years based on their employed design methodologies.

B. Graph Feature Enhancement

1) *Extracting Dependencies Between Features: AM-GCN*: The adaptive multi-channel graph convolution network (AM-GCN) [98] computes node similarity based on their features, constructing a k-nearest neighbor(k-NN) graph by selecting the k most analogous neighbors for each node. Then subsequently performs convolutions over both the k-NN graph and original graph. This design captures the dependencies between node features, thus achieving an enhancement of node features. For a k-NN graph $G_f = (\mathbf{A}_f, \mathbf{X})$, $\mathbf{A}_f$ denotes the adjacency matrix. And the message passing along G_f can be represented as (9).

$$\mathbf{h}_f^{(l+1)} = \text{RELU} \left(\tilde{\mathbf{D}}_f^{-1/2} \tilde{\mathbf{A}}_f \tilde{\mathbf{D}}_f^{-1/2} \mathbf{h}_f^{(l)} \mathbf{W}_f^{(l)} \right), \quad (9)$$

where $\mathbf{W}_f$ is the weight matrix, $\tilde{\mathbf{A}}_f = \mathbf{A}_f + \mathbf{I}_f$, and $\tilde{\mathbf{D}}_f$ is the diagonal degree matrix of $\tilde{\mathbf{A}}_f$.

CL-GNN: GNNs can extract more information regarding the local neighborhood dependencies of the nodes by sampling [103], while sampling the predicted node labels also allows GNNs to focus on the relationships between node features, graph topology, and label dependencies. [99] consider using a collective learning framework (CL-GNN), which employs Monte

TABLE II
SUMMARY OF THE POWERFUL MODEL

Methods	Subclasses	Models
Graph feature enhancement	Extracting dependencies between features	AM-GCN [98] CL-GNN [99] ACR-GNN [47]
	Enhancing feature utilization	MM-GNN [100]
Graph topology enhancement	Adding extra topology information	Twin-GNN [101] ID-GNN [61] CLIP [102] RP-GNN [103] RNI-GNN [104]
		DE-GNN [105] P-GNN [62] PEG [106] SMP [94] GD-WL [107]
	Encoding micro-topology	Eigen-GNN [108] SAGNN [109]
	Encoding global-topology	GSN [110] GraphSNN [111] GNN-AK [112] NGNN [113] MPSN [114] ESAN [115] LRP-GNN [64] k-GNN [34] K-hop GNN [116] KP-GNN [117] N2-FWL [118] PIN [119]
	Encoding local-topology	
GNNs architecture enhancement	Improving aggregation function	GIN [36] modular-GCN [120] diagonal-GNN [41] GNN-LF/HF [39] Geom-GCN [121] PG-GNN [122]
	Adopting equivariant architecture	k-IGN [69] PPGN [123] k-FGNN [124] Ring-GNN [37] SUN [125] GNNML [45] k-MPNN [70]

Carlo sampling to further consider label dependencies. This approach enables GNNs to incorporate more information into the estimated joint label distribution, leading to more expressive node embeddings.

ACR-GNN: Barceló et al. [47] adds a readout function in the aggregation operation to calculate the global node features for node update, resulting in a novel architecture known as aggregate-combine-readout GNN (ACR-GNN). The update process can be described as follows:

$$\mathbf{h}_v^{(l+1)} = \text{UPD}^{(l)} \left(\mathbf{h}_v^{(l)}, \text{AGG}^{(l)} \left(\left\{ \mathbf{h}_u^{(l)} : u \in N(v) \right\} \right), \right. \\ \left. \text{READ}^{(l)} \left(\left\{ \mathbf{h}_u^{(l)} : u \in V \right\} \right) \right). \quad (10)$$

2) *Enhancing Feature Utilization: MM-GNN*: Bi et al. [100] argued that most of the existing GNNs aggregating the features of the neighbors by a single statistic lose the information related to the feature distribution of the neighbors, and thus degrade the model performance. Inspired by the method of moments in statistical theory, they proposed to use multi-order moments to describe the distribution of neighborhood features, and based on this, they designed a new GNN model, i.e., a hybrid moment graph neural network (MM-GNN), by calculating the multiorder moments of each node's neighbors as a signature, and then assigning a larger weight to each node's significant moments and updating the node representations using an elemental-attention-based moment adapter. The update process can be described as follows:

$$a_k^{(l+1)}(v_i) = \sigma \left(\left[h_i^{(l)} \cdot W_{\text{query}}^{(l+1)} \parallel M_k^{(l+1)}(v_i) \cdot W_{\text{key}}^{(l+1)} \right] \cdot W_a^{(l+1)} \right), \quad (11)$$

$$h_i^{(l+1)} = \sum_{k=1}^K a_k^{(l+1)}(v_i) \odot M_k^{(l+1)}(v_i), a_k^{(l+1)}(v_i) \in \mathbb{R}^{1 \times D_{\text{hidden}}}, \quad (12)$$

where $W_a^{(l)}$ is the learnable attention matrix in l -th layer, $W_{\text{query}}^{(l)}$, $W_{\text{key}}^{(l)}$ are the transformation matrix for the query and key vectors in l -th layer. $M_k^{(l+1)}(v_i)$ is the form of k -th order moment for $(l+1)$ -th layer of GNNs. The $\odot$ is the element-wise product and K is the hyper-parameter for the largest ordinal of moments used in MM-GNN.

C. Graph Topology Enhancement

1) *Adding Extra Topology Information*: The aim of adding extra topology information is to increase the discrimination between nodes topologically. A prevalent and straightforward approach entails assigning distinct, recognizable features to nodes, such as identifications [101], [61], colors [102], and random features [93], [103], [104], [126], [127]. The added features can be represented by a characteristic matrix $\mathbf{C}$, as demonstrated in the (13), which enhances the node features $\mathbf{X}$ via $\mathbf{C}$. Especially, this method can also be applied to unfeatured graphs, in which case the additional features serve as node features.

$$G = (\mathbf{A}, \mathbf{X} \oplus \mathbf{C}). \quad (13)$$

Twin-GNN: Wang et al. [101] have introduced the Twin-WL test as an extension of the 1-WL test, with improved discriminative power obtained by simultaneously assigning color labels and identity id to the nodes and executing two sets of aggregation schemes. The identity aggregation as (14). Based on the Twin-WL test, the Twin-GNN has been developed to operate two sets of message passing mechanisms for node features and node identities. The identity passing encodes more complete topology information, allowing for a stronger expression than 1-WL test without significantly increasing computational costs.

$$\mathbf{id}_v^{(l)} = f_{id} \left(\mathbf{id}_v^{(l-1)}, \left\{ \mathbf{id}_u^{(l-1)} | u \in N(v) \right\} \right), \quad (14)$$

where f is a function able to convert the identity multiset to the topology information that can be compared across different graphs.

ID-GNN: The Identity-aware GNNs (ID-GNNs) [61] are a universal extension of GNNs. A L-layer ID-GNN first extracts the L-hop network $G_v^{(L)}$ of node v , and injects a unique color identity to v . Then, the heterogeneous message passing is performed on the $G_v^{(L)}$ to obtain the node embedding with injected identity. This added identification information enables the distinguishing of subgraphs with the same topology, thereby enhancing the GNNs expressive power.

For $u \in \mathcal{V}(G_v^{(L)})$, there can be categorized into two types throughout the embedding process: nodes with coloring and nodes without coloring. When passing the message of the colored nodes, perform another set of message passing mechanism:

$$\mathbf{m}_s^{(l)} = \text{MSG}_{\text{color}}^{(l)} \left(\left\{ \mathbf{h}_s^{(l-1)} : s \in N(u) \right\} \right). \quad (15)$$

CLIP: The GNN called Colored Local Iterative Procedure (CLIP) uses colors to disambiguate identical node features. Nodes with the same features are assigned to different subsets, and then the nodes in a same subset will be given a distinct color $\mathbf{c}_i$ from a finite color set $\mathcal{C}$. And the colored graph can be represented as the following Equation. The CLIP is capable of capturing topology characteristics that traditional MPNNs fail to distinguish, and becomes a provably universal extension of MPNNs.

$$G = (\mathbf{A}, \mathbf{X} \oplus [\mathbf{c}_i]). \quad (16)$$

RP-GNN: Murphy et al. [103] considered randomly assigning an order of nodes as their extra features and proposed the relational pooling GNN (RP) as a general method in GNNs. This method takes advantage of the enumeration idea, by enumerating all possible results of different input orders, and then taking the average result as the final output. Suppose there are all m possible permutations of graph G and for one permutation π , $\mathbf{I}_\pi$ is a one-hot encode matrix of the permutation sequence, $\pi \circ G = (\mathbf{A}, \mathbf{X} \oplus \mathbf{I}_\pi)$ denotes a permuted graph. Each permuted graph can get the node embeddings $\mathbf{h}_v^\pi$ via GNN, and take the average of them as the final embedding $\mathbf{h}_v$.

$$\mathbf{h}_v = \frac{1}{m!} \sum_m (\mathbf{h}_v^\pi). \quad (17)$$

RNI-GNN: The random node initialization (RNI) [104] is proposed as a means of training and implementing GNNs with randomized initial node features $\mathbf{R}$. RNI-GNNs are shown to be effective in achieving universal without relying on high-order feature calculations. This also holds with partially randomized initial node features.

$$G = (\mathbf{A}, \mathbf{X} \oplus \mathbf{R}). \quad (18)$$

2) *Encoding the Micro-Topology:* Encoding the micro-topology is to determine the position of each node within the graph topology, accomplished through distance encoding or position encoding.

DE-GNN: The distance encoding (DE) [105] is proposed as a generic initialization mean of contributing to the node embeddings in GNNs by capturing the distance from a given node to the node set to be learnt. Given a node set S whose topology representation is to be learnt, for every node u in the graph, DE is defined as a mapping f_{DE} of a set of landing probabilities of random walks from each node of the set S to node u . DE-GNN is proved to be able to effectively distinguish node sets within nearly all regular graphs where traditional GNNs struggle. While it introduces significant memory and time overhead.

$$\mathbf{d}_{v|S} = \text{AGG}(f_{\text{DE}}(v, u) | u \in S), \quad (19)$$

$$\mathbf{h}_v^{(0)} = \mathbf{x}_v \otimes \mathbf{d}_{v|S}, \quad (20)$$

where $\mathbf{d}_{v|S}$ denotes the distance of node v and set S , $\otimes$ is the concatenation operation.

P-GNN: In order to mitigate the computational cost of DE, an approach called position encoding (PE) is applied using the absolute position of the nodes in the graph as an additional feature. This technique approximates DE by measuring the distances between position features and can be shared among different node sets. The position-aware GNN (P-GNN) proposed by You et al. [62] incorporates PE by measuring the distance from a node to a set of randomly selected anchor nodes. However, it has been observed that this approach has slow convergence rate and unsatisfactory performance. The procedure of PE is shown as following:

$$\mathbf{d}_{v,u}^q = \begin{cases} \mathbf{d}_{v,u}, & \mathbf{d}_{v,u} \leq q \\ \infty, & \text{otherwise,} \end{cases} \quad (21)$$

$$\mathbf{d}_{\text{standard}} = \frac{1}{1 + \mathbf{d}_{v,u}^q}, \quad (22)$$

where q is the hop from v to u , $\mathbf{d}_{v,u}$ is the shortest distance of v and u .

PEG: Utilizing PE, [106] introduces a novel approach to constructing more powerful GNNs - a class of GNN layers termed PEG. PEG uses separate channels to update both the node features and position features, exhibiting permutation equivariance and rotation equivariance simultaneously, where Stability is actually an even stronger condition than permutation equivariance because it characterizes how much gap between the predictions of the two graphs is expected when they do not perfectly match each other.

In addition to DE or PE, incorporating marked topology information into message passing mechanisms shows significant enhancement of the GNNs expressive power.

SMP: The topology message passing (SMP) [94] method employs one-hot encoding to propagate the sole node identification, enabling the learning of local context matrices surrounding each node. This confers SMP with the capability to learn rich local topological information. In SMP, each node of a graph maintains a local context matrix $\mathbf{U}_i$ rather than a feature vector $\mathbf{x}_i$ as in MPNN. The $\mathbf{U}_i$ is initialized as a one-hot encoding $\mathbf{U}_i^{(0)} = \mathbf{1}$ for every $v_i \in \mathcal{V}$, and if there are features $\mathbf{x}_i$ associated with node v_i , they are appended to the same row of the local context as $\mathbf{U}_i^{(0)}[i, :] = [1, \mathbf{x}_i]$. The message passing process is as (23)

showed.

$$\mathbf{U}_i^{(l+1)} = \text{UPD}(\mathbf{U}_i^l, \text{AGG}(\text{MSG}(\mathbf{U}_j^l)_{v_j \in N(v_i)})). \quad (23)$$

GD-WL: One of the main drawbacks of 1-WL is the lack of distance information between nodes. Inspired by some distance coding methods [62], [94], [106], [107] further introduced a new color refinement framework called Generalized Distance Weisfeller-Lehman (GD-WL). The update rule of GD-WL is very simple and can be written as:

$$\mathbf{c}_v^{(l)} = \text{Hash} \left(\mathbf{c}_v^{(l-1)}, \left\{ \left\{ \left(d_{vu}, \mathbf{c}_u^{(l-1)} \right) \mid u \in N(v) \right\} \right\} \right), \quad (24)$$

where d_{vu} can be an arbitrary distance metric.

3) *Encoding the Local-Topology:* This method focuses on local topology, mainly utilizing subgraphs explicitly or implicitly for node embedding to learn more topology information.

GSN: Graph sub-topology Networks (GSN) [110] represent a topologically-aware message passing scheme based on sub-topology encoding. GSN explicit encodes the count of pre-defined subgraph topology appearance in the graph, as a topology role in message passing to capture a richer topological features and thereby exhibit superior expressive power over the WL test. However, the pre-defined subgraphs require the domain expert knowledge, hampering the model's widespread adoption. Considering $\mathbf{r}_v$ as the topology role induced by the subgraph counts, the message passing process is as following:

$$\mathbf{m}_v^{(l+1)} = \text{MSG}(\mathbf{h}_v^{(l)}, \mathbf{h}_u^{(l)}, \mathbf{r}_v, \mathbf{r}_u : u \in N(v)). \quad (25)$$

GraphSNN: Wijesinghe et al. [111] introduced the neighborhood subgraph graph neural network (GraphSNN), which enables the transmission of diverse messages based on the topology features of the central node's 1-hop neighborhood subgraph during message passing. The weight of the delivered message is determined by the extent of overlap subgraph between the central node and its neighbors in (26) and (27), with edge overlap also factored into the weight calculation. By injecting the topology information of the neighborhood subgraph in the message passing process as (28), GraphSNN can improve its expressive power without necessitating specialized knowledge or sacrificing computational complexity.

$$w_{vu} = \frac{|E_{vu}|}{|V_{vu}| \cdot |V_{vu} - 1|} \cdot |V_{vu}|^\lambda, \quad (26)$$

$$\mathbf{W} = \left(\frac{w_{vu}}{\sum_{u \in N(v)} w_{vu}} \right)_{v, u \in V}, \quad (27)$$

$$\begin{aligned} \mathbf{h}_v^{(l+1)} = & \text{MLP}_\theta \left(\gamma^{(l)} \left(\sum_{u \in N(v)} \mathbf{W}_{vu} + 1 \right) \mathbf{h}_v^{(l)} \right. \\ & \left. + \sum_{u \in N(v)} (\mathbf{W}_{vu} + 1) \mathbf{h}_u^{(l)} \right), \end{aligned} \quad (28)$$

where $\lambda > 0$ and the overlap subgraph is denoted as $\mathcal{G}_{vu} = (V_{vu}, E_{vu}) = S_v \cap S_u$ for neighboring nodes v, u . $\mathbf{W}$ is the weight matrix. γ and θ is learnable scalar parameter.

GNN-AK [112] extends the scope of local aggregation in GNNs beyond 1-hop neighbors (star-like) into more generalized subgraphs. This method, termed as GNN-AK, computes the encoding of induced subgraphs of nodes and serves as a wrapper to uplift any GNN, where GNN perform as a kernel. Let $N_k(v)$ be the set of nodes in the k-hop egonet rooted at node v , containing node v . For $N_k(v) \subseteq \mathcal{V}$, $G[N_k(v)]$ is the induced subgraph of v : $G[N_k(v)] = (N_k(v), (i, j) \in \mathcal{E} \mid i, j \in N_k(v))$. Every subgraph $G[N_k(v)]$ is encoded as $\mathbf{g}_{\text{sub}(v)}$, concatenating with the node embedding $\mathbf{h}_{\text{sub}(v)}$ aggregated from $N_k(v)$, to obtain the final embedding $\mathbf{h}_v$:

$$\mathbf{g}_{\text{sub}(v)}^{(l+1)} = \text{POOL}^{(l)}(\mathbf{h}_{\text{sub}(i)} \mid i \in N_k(v)), \quad (29)$$

$$\mathbf{h}_v = \text{CON}(\mathbf{g}_{\text{sub}(v)}, \mathbf{h}_{\text{sub}(v)}). \quad (30)$$

NGNN: Similar to the design philosophy of GNN-AK, Nested Graph Neural Networks (NGNNs) [113] extract a local subgraph around each node and apply a GNN to learning the representation of each subgraph. The overall graph representation is obtained by aggregating these subgraph representations. However, NGNN differs from GNN-AK in subgraph design. Specifically, the height of the rooted subgraph in NGNN can be determined based on the given hop of the node.

MPSN: The Message Passing Simplex Networks (MP-SNs) [114], which operates message passing on simplicial complexes (SCs) [128]. SCs are in correspondence with subgraph topology, where SCs of varying dimensions concretize topology information of graph at different levels. Specifically, 0-simplices can be interpreted as vertices, 1-simplices as edges, 2-simplices as triangles, and so on.

ESAN: An empirical observation is that although two graphs may not be discriminable by MPNNs, they generally contain distinguishable subgraphs. Thus, [115] recommends representing each graph as a set of subgraphs derived from a predefined policy and proposes a novel equivariant subgraph aggregation network (ESAN) framework for handling them, which comprises the DSS-GNN architecture and subgraph selection policy. The core of the approach is to represent the graph G as a bag (multiset) $S_G = \{\{G_1, \dots, G_m\}\}$ of its subgraphs, and to make a prediction on a graph based on this subset. Motivated by the linear characterization, DSS-GNN adopt the same layer topology:

$$F_{\text{DSS-GNN}} = E_{\text{sets}} \circ R_{\text{subgraphs}} \circ E_{\text{subgraphs}}, \quad (31)$$

$$E_{\text{subgraphs}} : (L(\mathbf{A}, \mathbf{X}))_i = L^1(\mathbf{A}_i, \mathbf{X}_i) + L^2 \left(\sum_{j=1}^m \mathbf{A}_j, \sum_{j=1}^m \mathbf{X}_j \right), \quad (32)$$

where $E_{\text{subgraphs}}$ is an equivariant feature encoding layer, $R_{\text{subgraphs}}$ is a subgraph readout layer, E_{sets} is a universal set encoding layer. And $(L(\mathbf{A}, \mathbf{X}))_i$ denotes the output of the layer on the i-th subgraph, L^1, L^2 represent two graph encoders and can be any type of GNNs layer.

The methods outlined above pertain primarily to explicit utilization of subgraph topology, however implicit utilization can also serve to enhance the learning of graph topology.

LPR-GNN: Inspired by RP-GNN [103] and subgraph counting task, the Local Relational Pooling model (LRP-GNN) [64] is proposed to achieve competitive performance on sub-topology counting. LRP-GNN refers to RP-GNN that operates on subgraphs essentially, but in comparison to previous works [110], [111], [112], LRP can not only compute sub-topology, but also learn they relevant relation of sub-topology.

k-GNN: The higher-order message passing (k-GNN) scheme, implicitly utilizes subgraphs. The k-WL test serves as the foundation for k-GNNs. Whereas the standard WL test iteratively aggregates neighbors of a single node, k-WL operates on tuples of k-nodes for iterative aggregation. These k-tuples can be interpreted as subgraphs, where higher-order message passing occurs between subgraphs. Equation (33) redefines the neighborhood representation of k-tuples, and (34) enables reliable capture of inherent local graph topology through information propagation and embedding update over k-neighborhoods. It has been proved that the discrimination power of 1-WL is equivalent to 2-WL. Moreover, for $k \geq 2$, (k+1)-WL is strictly more powerful than k-WL. Notably, 3-WL is considered to be the strongest expressive power to identify all non-isomorphic graphs to date [35], [129]. The k-GNNs expressive power is equal to k-WL, hence when $k \geq 2$, k-GNNs offers the strongest expression rivaled to 3-WL.

$$N(s) = \{t \in [V]^k \mid |s \cap t| = k-1\}, \quad (33)$$

$$\mathbf{h}_k^{(l+1)}(s) = \sigma \left(\mathbf{h}_k^{(l)}(s) \cdot \mathbf{W}_1^{(l+1)} + \sum_{u \in N(s)} \mathbf{h}_k^{(l)}(u) \cdot \mathbf{W}_2^{(l+1)} \right), \quad (34)$$

where all subsets of size k over the set $\mathcal{V}$ are considered as $[V]^k$, and a k -set in $[V]^k$ is denoted by $s = \{v_1, \dots, v_k\}$ as the central tuple. Moreover, $N(s)$ denotes the neighborhood of s , and $\mathbf{W}_1, \mathbf{W}_2$ are the parameter matrix.

K-hop GNN: In MPNN, the representation of a node is updated by aggregating its direct neighbours, which are called 1-hop neighbours. Some work extends the message passing to the concept of [116], [130], [131], [132], [133], [134] K-hops. In K-hop message passing, the node representation is updated not only by aggregating the information of the node's 1-hop neighbours, but also by aggregating all the neighbours within the node's K-hop. The K-hop neighbours of node v refer to all neighbours whose distance from node v is less than or equal to K. Denote by $N(v)^k$ the K-th hop neighbour of node v , i.e., the neighbour whose distance to the node is exactly equal to K.

$$\mathbf{m}_v^{(l,k)} = \text{MSG}^{(l)} \left(\left\{ \left\{ \mathbf{h}_u^{(l-1)} : u \in N(v)^k \right\} \right\} \right), \quad (35)$$

$$\mathbf{h}_v^{(l,k)} = \text{UPD}^{(l)} \left(\mathbf{m}_v^{(l,k)}, \mathbf{h}_v^{(l-1)} \right), \quad (36)$$

$$\mathbf{h}_v^{(l)} = \text{AGG}^{(l)} \left(\left\{ \left\{ \mathbf{h}_v^{(l,k)} \mid k = 1, 2, \dots, K \right\} \right\} \right). \quad (37)$$

The K-hop message passing is more powerful than 1-WL and can distinguish almost all rule-graphs, but its expressive power is similarly limited by 3-WL.

KP-GNN: Introduced K-hop peripheral-subgraph-enhanced GNN (KP-GNN) [117], a new GNN framework with k-hop message passing, which significantly improves the expressive power of K-hop GNN. KP-GNN aggregates in the neighbour aggregation of each hop not only the neighbouring nodes, but also aggregates peripheral subgraphs (subgraphs induced by neighbours in that hop). This additional information helps KP-GNN learn more expressive local topology information around nodes. This improvement occurs in the message passing process:

$$\mathbf{m}_v^{(l,k)} = \text{MSG}^{(l)} \left(\left\{ \left\{ \mathbf{h}_u^{(l-1)} : u \in N(v)^k \right\} \right\}, G_v^k \right), \quad (38)$$

where G_v^k is subgraphs induced by neighbours of v in hop k .

N2-FWL extends the traditional k-FWL algorithm by constructing a neighbourhood tuple [118]. This neighbourhood tuple contains all possible combinations of node substitutions, which enables to capture richer information about the graph structure. By considering a wider range of neighbourhood information, N2-FWL is able to capture higher-order structures in the graph, such as 6-rings, 4-connected subgraphs, and 4-paths, which improves the expressive power of the model. In N2-FWL, the color update rule for node pair (v_1, v_2) can be expressed as:

$$\hat{C}_l^{(2,2)}(v_1, v_2) = \text{HASH} \left(\hat{C}_{l-1}^{(2,2)}(v_1, v_2), \left\{ \hat{C}_{l-1}^{(2,2)}(u_1, u_2) \mid (u_1, u_2) \in \mathcal{Q}_F^{(w_1, w_2)}(v_1, v_2) \right\} \right), \quad (39)$$

where $\mathcal{Q}_F^{(w_1, w_2)}(v_1, v_2)$ denotes the neighbourhood tuple based on the node pairs (w_1, w_2) , and $\hat{C}_l^{(2,2)}$ denotes the color of the node pairs in the first iteration. N2-FWL is able to capture the higher-order structure of graphs, has theoretically comparable expressive power to 3-FWL, and can be designed with different instances according to different task requirements. However, it may face the problem of computational complexity in practical applications, especially when dealing with large graphs.

PIN: The Path Isomorphism Network (PIN) [119] implements the Path Weisfeiler-Lehman (PWL) test by lifting the graph-to-path complex, using simple paths as base elements, and updating the feature representation based on part-whole and adjacency relationships through a message passing process. This approach does not rely on the assumption of graph substructures (e.g., clusters, rings, etc.), but focuses on simple paths of the graph. In this way PIN enhances the understanding of the graph structure and thus improves its expressive power. In the PWL test, for each time step t and each simple path σ , a hash function is used to update the colors based on the current color and the associated set of colors:

$$ct_\sigma^{(t+1)} = \text{HASH} \left(ct_\sigma^{(t)}, ct_{B(\sigma)}^{(t)}, ct_{\uparrow(\sigma)}^{(t)}, ct_{\downarrow(\sigma)}^{(t)} \right), \quad (40)$$

where $ct_\sigma^{(t)}$ is the color of the path σ at time step t , $B(\sigma)$ is the set of boundaries of σ , and $\uparrow(\sigma)$ and $\downarrow(\sigma)$ are the set of up-neighbours and down-neighbours of σ , respectively. The final color of the path complex $\mathcal{P}$ is represented using the set of colors of all simple paths.

4) *Encoding the Global-Topology*: Encoding the global-topology is to directly extract the topology information of the whole graph for learning.

Eigen-GNN: Zhang et al. [108] introduced the topology eigen decomposition module, designed as a universal plug-in to GNNs termed as Eigen-GNN. This approach incorporates eigen decomposition of the adjacency matrix $\mathbf{A}$ of the graph to obtain eigenvectors as topology features, which are subsequently concatenated with node features. The resulting amalgamation is then utilized as input to GNNs, as denoted in (41).

$$\mathbf{H}^{(0)} = [\mathbf{X}, f(\mathbf{Q})], \quad (41)$$

where $\mathbf{Q}$ is the eigenvectors corresponding to the top-d largest absolute eigenvalues of $\mathbf{A}$, f is a function such as normalization or identity mapping.

SAGNN: Zeng et al. [109] enhances the model's ability to perceive the graph structure by innovatively introducing the subgraph encoding and information injection mechanism, which captures the local structural information around the nodes by Ego subgraph and the global structural information of the whole graph by Cut subgraph, and then injects the structural information encoded by the subgraph into the message passing process of the GNNs. SAGNN obtains theoretically more powerful expressive power than 1-WL, but may face the problem of higher computational cost in practical applications, especially when dealing with large graphs.

D. GNNs Architecture Enhancement

1) *Improving Aggregation Functions*: Improving aggregation functions involves improvements to the aggregation operations, wherein most of the modifications are made while retaining the permutation invariance of the aggregation function. However, there exist a few exceptions to break the permutation invariance. The improvement of aggregator that preserves invariance is established based on the theory proposed by Xu et al. [36].

GIN: The essence of MPNN is actually a function that maps multiple set, where allow repeated elements to exist. The capacity of multi-set functions to discern these repeated elements inherently constrains the expressive power of MPNN. As a result, optimizing the aggregation function necessitates the development of an injective function operating on multisets, ensuring distinct embeddings for different nodes. The graph isomorphism network (GIN) is a simple architecture to satisfy this prerequisite:

$$\mathbf{h}_v^{(l+1)} = \text{MLP} \left(\left(1 + \varepsilon^{(l)}\right) \mathbf{h}_v^{(l)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(l)} \right), \quad (42)$$

where $\varepsilon^{(l)}$ represents the learnable or fixed scalar and the MLP plays a critical role in strengthening the expressive power by ensuring the injective property.

modular-GCN: The global parameter assignment strategy enforced by GCN is actually a permutation invariant aggregation operation. Dehmamy et al. [120] has insight into the loss of

topology information in GCN, and designed three GCN modules using three different propagation strategies as (1) $f_1 = \mathbf{A}$, (2) $f_2 = \mathbf{D}^{-1} \mathbf{A}$, and (3) $f_3 = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ to address this limitation. Combining the three GCN modules compensates for the loss of missing topology information, realizing the optimization of the aggregation function.

Furthermore, the general aggregation operations in GNNs are interpreted as graph convolutions. The graph topology acts as a node feature filter, smoothing node features during the convolution process, which corresponds to a filtering operation. Thereby enable the analysis of GNNs expressive power through spectral decomposition techniques [40].

diagonal-GNN: Kanatsoulis et al. [41] utilizes spectral decomposition tools to view GNNs as graph feature filters, as shown in (43), and proposed a GNN designed with the diagonal modules (diagonal-GNN). The use of diagonal modules allows GNNs to capture broader topology relationships while also capturing neighborhood relationships, enabling GNNs show the equivalent recognition abilities of featured graphs even on unfeatured graphs.

$$\mathbf{H} = \sigma \left(\sum_{k=0}^{K-1} \mathbf{A}^k \mathbf{X} \mathbf{W}_k \right), \quad (43)$$

$$\text{diagonal-GNN} : \mathbf{H} = \sigma \left(\sum_{k=0}^{K-1} \text{diag}(\mathbf{A}^k) \mathbf{X} \mathbf{W}_k \right), \quad (44)$$

where K is the length of graph filter, σ denotes a nonlinearity, $\mathbf{W}_k$ is the filter parameters and can be a matrix, a vector, or a scalar.

GNN-LF/HF: Leveraging spectral decomposition tools to design various convolution processes to handle specific filtering conditions, GNNs can recognize and process more complex low-frequency or high-frequency information. [39] considers two distinct modes of low-pass filtering (LF) and high-pass filtering (HF), develop corresponding convolution operations, and design new models GNN-LF/HF respectively. The propagation mechanism of GNN-LF can be expressed by the following equations:

$$\begin{aligned} \mathbf{H}^{(l+1)} = & f_{\text{COM}} \left(\frac{1 + \alpha\mu - 2\alpha}{1 + \alpha\mu - \alpha} \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \right. \\ & \left. + \frac{\alpha\mu}{1 + \alpha\mu - \alpha} \mathbf{X} + \frac{\alpha - \alpha\mu}{1 + \alpha\mu - \alpha} \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{X} \right), \end{aligned} \quad (45)$$

$$\mathbf{H}^{(0)} = \frac{\mu}{1 + \alpha\mu - \alpha} \mathbf{X} + \frac{1 - \mu}{1 + \alpha\mu - \alpha} \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{X}. \quad (46)$$

The propagation mechanism of GNN-HF is given by the following equations:

$$\begin{aligned} \mathbf{H}^{(l+1)} = & f_{\text{COM}} \left(\frac{\alpha\beta - \alpha + 1}{\alpha\beta + 1} \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \right. \\ & \left. + \frac{\alpha}{\alpha\beta + 1} \mathbf{X} + \frac{\alpha\beta}{\alpha\beta + 1} \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{X} \right), \end{aligned} \quad (47)$$

$$\mathbf{H}^{(0)} = \frac{1}{\alpha\beta + 1} \mathbf{X} + \frac{\beta}{\alpha\beta + 1} \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{X}, \quad (48)$$

where μ, α, β are hyper parameters that control the trade-off between the topology and feature information at different scales. $\tilde{\mathbf{A}}$ and $\tilde{\mathbf{D}}$ are the normalized adjacency matrix and the diagonal degree matrix, respectively, that account for the topology of the graph data.

Geom-GCN: Pei et al. [121] leverage the network geometry theory to propose a geometric aggregation scheme, implemented within graph convolution networks as Geom-GCN. This scheme maps graphs to a continuous latent space via node embeddings and subsequently constructs topology neighborhoods based on the geometric relations defined in the latent space, effectively preserving the intricate topological patterns present within the graphs. Building upon the definition of the original neighborhood, they further define topology neighborhoods $\mathcal{N}(v) = (\{N_g(v), N_s(v)\}, \tau)$, consisting of a set of neighborhood $\{N_g(v), N_s(v)\}$ and a relation operator on neighborhoods τ . $N_g(v) = \{u | u \in \mathcal{V}, (u, v) \in \mathcal{E}\}$ is the neighborhood in the graph. $N_s(v) = \{u | u \in \mathcal{V}, d(\mathbf{z}_u, \mathbf{z}_v) < \rho\}$ is the neighborhood in the latent space, where d is the distance function, ρ is the pre-given parameter, and $\mathbf{z}_v \in \mathbf{R}$ can be considered as the position of nodes in the latent space via the mapping $f : v \rightarrow \mathbf{z}_v$. And $\tau : (\mathbf{z}_v, \mathbf{z}_u) \rightarrow r \in R$, where R is the set of the geometric relationships and r can be specified as an arbitrary geometric relationship of interest according to the particular latent space and applications. On the basis, a bi-level aggregation scheme is introduced:

$$\mathbf{m}_{(i,r)}^{v,l+1} = \text{AGG}_1(\{\mathbf{h}_u^l | u \in N_i(v), \tau(\mathbf{z}_u, \mathbf{z}_v) = r\}), \quad (49)$$

$$\mathbf{m}_v^{l+1} = \text{AGG}_2((\mathbf{m}_{(i,r)}^{v,l+1}, (i, r))), \quad (50)$$

where $\mathbf{m}_{(i,r)}^{v,l+1}$ can be considered as the feature of virtual nodes in the latent space which is indexed by (i, r) which is corresponding to the combination of a neighborhood i and a relationship r .

The approach of optimizing the permutation invariant aggregation function presents significant limitations. Therefore, alternative designs have been proposed to address this constraint.

PG-GNN: Permutation sensitive functions are regarded as a symmetry breaking mechanism that disrupts the equidistant status of adjacent nodes, allowing for the capture of relationships between neighboring nodes. However, few permutation-sensitive GNNs [11], [102], [135] are available to real-world applications, due to their complexity. Huang et.al. [122] adopts Recurrent neural networks (RNNs) to model the dependency relationships between neighboring nodes as a permutation-sensitive function approach, wherein all permutations π act on the $\mathcal{V}$, leading to the following aggregation strategy:

$$\mathbf{h}_v^{(l+1)} = \sum_{\pi \in \Pi} \text{RNN}(\mathbf{h}_{\pi(u_1)}^{(l)}, \mathbf{h}_{\pi(u_2)}^{(l)}, \dots, \mathbf{h}_{\pi(u_n)}^{(l)}) + \mathbf{W}^{(l)} \mathbf{h}_v^{(l)}, \quad (51)$$

where $u_{1:n} \in N(v)$, $\mathbf{W}$ is the parameter matrix.

2) **Adopting Equivariant Architecture:** The bottleneck of GNNs expressive power lies in the permutation invariance of their aggregation operators [93]. While the limitation only serves

as a condition to induce permutation equivariance, rather than a requisite component in GNN design. Consequently, abandoning aggregation operations and employing distinct arrangements of equivariant functions or operators can lead to the direct design of equivariant GNNs [26], [27], [37], [45], [69], [70], [123], [124], [125]. However, it is worth noting that equivariant GNNs perform calculations on global features, demanding substantial memory and computational resources, rendering these networks primarily as theoretical tools with limited practical applicability. Hence, the following section will offer a succinct introduction to several equivariant GNNs models as a supplement to the method of improving GNNs expressive power.

k-IGN: Maron et al. [25], [69] develop the k-order linear invariant graph neural network (k-IGN), which operates calculation directly on global information tensors. K-IGN comprises of a sequence of alternating layers composed of invariant or equivariant linear layers and nonlinear activation layers and demonstrates a strong expressive power not less than MPNNs, equivalent to the k-WL test [124].

The provably powerful graph networks (PPGN) [123], the folklore graph neural network (FGNN) [124], [26] and the Ring-GNN [37] are improvements to k-IGN, and by simplifying the computational process, they can achieve 3-WL expressive power at $k = 2$.

SUN: Inspired by a series of subgraph GNNs [104], [136], [137], [138], Frasca et al. [125] proposes a novel subgraph learning framework based on 2-IGN, termed as the Subgraph Union Network (SUN). Rather than improving the computational process, SUN improves its expression by optimizing the 2-IGN layer to capture local topology information.

The mentioned models are designed based on the IGN framework, which utilizes arrangements of equivariant functions to obtain equivariant networks. The following will introduce some designs that obtain equivariant networks by arranging equivariant operators [44], [45], [46], [47], [48], [70].

These models are formulated using matrix language (ML) or tensor language (TL) to describe operators, which are then combined in various ways to obtain GNNs with different expressive powers. Balcilar et al. [45] employ equivariant operators in ML to design two models, *GNNML1* and *GNNML3*, with 1-WL and 3-WL expressive powers, respectively. Meanwhile, Geerts et al. [70] utilize the ML to formulate *k-MPNN*.

These higher-order equivariant networks are designed to improve the expressive power by the inspiration of the k-WL hierarchy. Ideally, we could improve the expression of the models by increasing k in the k-WL hierarchy. However, in fact, due to the combinatorial growth of complexity, these models can only be applied in practice [25], [34], [123], [139] to small graphs with small k . Specifically the limit of practical application is reached at $k = 3$. Their poor scalability becomes a key limitation for the practical application of such networks.

E. Comparison and Analysis of Methods

1) **Advantages and Drawbacks of Methods:** Graph feature enhancement enhances the model's understanding of node/edge

features, beneficial for tasks with complex feature relationships, like various tasks on heterogeneous graphs.

Extracting feature dependencies is suitable for tasks where feature relationships significantly influence performance. However, introducing complex dependencies can lead to overfitting and raise computational demands.

Enhancing feature utilization can improve the interpretability and accuracy of classification results, crucial for applications requiring a thorough understanding of feature distributions. However, the introduction of random features can disrupt permutation invariance, necessitating additional design considerations [103] to maintain this property.

Graph topology enhancement strengthens the model's comprehension of graph structures, particularly in sparse-feature contexts. However, it requires domain knowledge for feature selection, and potentially escalate memory and computational expenses, hindering scalability on large datasets.

Adding extra topology information is beneficial in sparse-feature scenarios, commonly used in social network analysis and recommendation systems, but often require domain knowledge for effective feature selection.

Encoding micro-topology is critical for tasks requiring precise, relative or absolute positional information, yet introduces high implementation complexity and additional memory and computational costs.

Encoding local-topology is effective for tasks dependent on local structural features, such as community detection and network security, but it can be computationally demanding and require expert knowledge.

Encoding global-topology is essential for tasks needing comprehensive graph understanding, such as graph classification and network traffic analysis, but overemphasis on global information risks information overload.

GNNs architecture enhancement enables the model to learn complex graph structures more effectively. However, the high memory and computational demands restrict its application to large datasets, and the addition of hyper-parameters complicates the model.

Improving aggregation functions is suitable for tasks requiring specific topological pattern recognition and spectral analysis, but increases computational load and necessitates meticulous hyper-parameter tuning.

Adopting equivariant architectures provides robust theoretical expressivity, comparable to the k-WL test, often serving as a theoretical tool for understanding GNNs expressive power. However, due to their reliance on global feature computation, these architectures demand substantial memory and computational resources, limiting their scalability to large datasets.

2) *Assessment of Some Existing Models:* Tables III and IV present a comparative analysis of model performance across node classification and graph classification tasks using three real-world datasets separately: Cora, Citeseer, PubMed for node classification, and ZINC, MUTAG, PROTEINS for graph classification. Metrics include MAE for graph classification on ZINC dataset and accuracy for other tasks. The analysis encompasses a range of GNN baselines, including GCN [140], GAT [3],

TABLE III
PERFORMANCE OF VARIOUS GNN MODELS ON NODE CLASSIFICATION DATASETS

Model	Cora	Citeseer	PubMed
GCN [140]	0.811 ± 0.003	0.701 ± 0.004	0.786 ± 0.006
GAT [3]	0.814 ± 0.010	0.709 ± 0.007	0.782 ± 0.008
GraphSAGE [1]	0.804 ± 0.004	0.696 ± 0.003	0.775 ± 0.008
AM-GCN [98]	-	<u>0.745 ± 0.010</u>	-
CL-GNN [99]	0.748 ± 0.001	-	0.765 ± 0.002
MM-GNN [100]	0.842 ± 0.006	0.730 ± 0.006	0.803 ± 0.007
ID-GNN [61]	<u>0.870 ± 0.220</u>	0.742 ± 0.010	-
PEG [106]	0.822 ± 0.001	0.719 ± 0.001	0.799 ± 0.001
Eigen-GNN [108]	0.789 ± 0.007	0.665 ± 0.003	0.786 ± 0.001
GraphSNN [111]	0.838 ± 0.012	0.735 ± 0.016	0.796 ± 0.014
GIN [36]	0.768 ± 0.009	0.677 ± 0.003	0.770 ± 0.005
GNN-LF [39]	0.837 ± 0.001	0.720 ± 0.003	0.803 ± 0.002
GNN-HF [39]	0.840 ± 0.002	0.723 ± 0.003	<u>0.804 ± 0.003</u>
Geom-GCN [121]	<u>0.854 ± 0.004</u>	<u>0.760 ± 0.015</u>	<u>0.879 ± 0.025</u>

This table compares the performance of various GNNs on three different node classification datasets (Cora, Citeseer, PubMed), using accuracy as metric, categorized by methods that enhance the model's expressive power. Best: underlined; second-best: dashed lines.

TABLE IV
PERFORMANCE OF VARIOUS GNN MODELS ON GRAPH CLASSIFICATION DATASETS

Model	ZINC	MUTAG	PROTEINS
GCN [140]	0.321 ± 0.009	0.891 ± 0.058	0.752 ± 0.051
GAT [3]	0.384 ± 0.007	0.901 ± 0.058	0.759 ± 0.043
GraphSAGE [1]	0.384 ± 0.007	0.851 ± 0.076	0.692 ± 0.100
Twin-GNN [101]	-	0.900 ± 0.039	-
ID-GNN [61]	0.083 ± 0.003	0.904 ± 0.054	0.754 ± 0.027
CLIP [102]	-	0.939 ± 0.040	0.771 ± 0.044
GD-WL [107]	0.081 ± 0.009	-	-
SAGNN [109]	0.072 ± 0.002	<u>0.952 ± 0.030</u>	<u>0.798 ± 0.038</u>
GSN [110]	0.101 ± 0.010	0.922 ± 0.075	0.766 ± 0.050
GraphSNN [111]	-	0.947 ± 0.019	0.784 ± 0.027
GNN-AK [112]	0.105 ± 0.010	0.913 ± 0.070	0.771 ± 0.057
NGNN [113]	0.111 ± 0.003	0.879 ± 0.082	0.742 ± 0.037
MPSN [114]	0.079 ± 0.006	-	0.764 ± 0.033
ESAN [115]	0.102 ± 0.003	0.911 ± 0.070	0.759 ± 0.043
LRP-GNN [64]	0.223 ± 0.008	-	-
3-GNN [34]	0.407 ± 0.028	0.861 ± 0.001	0.755 ± 0.001
3-hop GNN [116]	-	0.876 ± 0.007	0.753 ± 0.004
KP-GNN [117]	-	<u>0.961 ± 0.046</u>	<u>0.801 ± 0.043</u>
N2-FWL [118]	<u>0.059 ± 0.001</u>	-	-
PIN [119]	0.096 ± 0.003	-	0.788 ± 0.044
GIN [36]	0.387 ± 0.015	0.894 ± 0.056	0.762 ± 0.028
PG-GNN [122]	0.282 ± 0.011	-	0.768 ± 0.038
IGN [69]	<u>0.045 ± 0.006</u>	0.839 ± 0.130	0.766 ± 0.055
PPGN [124]	0.159 ± 0.007	0.906 ± 0.087	0.772 ± 0.047
Ring-GNN [37]	0.353 ± 0.019	0.868 ± 0.064	0.757 ± 0.029
SUN [125]	0.083 ± 0.003	0.727 ± 0.058	0.758 ± 0.050
GNNML1 [45]	0.314 ± 0.015	0.900 ± 0.043	0.758 ± 0.049
GNNML3 [45]	0.161 ± 0.006	0.909 ± 0.055	0.764 ± 0.051
k-MPNN [70]	0.145 ± 0.007	-	-

This table compares the performance of various GNNs on three different graph classification datasets (ZINC, MUTAG, PROTEINS), using MAE(ZINC) and accuracy(MUTAG, PROTEINS) as metrics, categorized by methods that enhance the model's expressive power. Best: underlined; second-best: dashed lines.

and GraphSAGE [1], with experimental results sourced from scholarly literature.

This table indicates that graph feature enhancement is primarily used for node classification, while graph topology enhancement is better suited for graph classification. GNN architecture enhancement is versatile for both tasks. Model performance varies significantly across in different tasks, with excellence in one not guaranteeing proficiency in others, highlighting the need for method selection tailored to dataset characteristics and task requirements. For node classification, improving aggregation functions improves expressiveness and accuracy. For graph classification, encoding global-topology is preferred. Additionally, computational efficiency and model scalability are crucial considerations in method choices.

V. CHALLENGE AND FUTURE DIRECTIONS

Refinement of Assessment Indicators: Current research, such as by Zhang et al. [91], has introduced quantitative criteria to assess GNN expressive power, an improvement over the initial WL tests. These criteria enhance model scalability and reduce abrupt task shifts [141]. However, they only cover topological representation, neglecting feature embedding ability, which is known to interact positively with topology. Thus, there's a need for further refinement of these metrics to include comprehensive evaluation of both feature embedding and topological representation, with the expectation of seeing such metrics developed soon.

More GNNs Architecture Design: Message passing in GNNs has seen widespread success [142] but is limited by the WL test in expressive power. Alternatives like LGNN face development hurdles due to tensor calculation complexities. Future research should focus on designing more expressive GNN architectures, possibly incorporating multi-level messaging and multi-scale graph embedding to capture complex graph features. Integrating GNNs with physical models, such as molecular dynamics for improved molecular structure and property capture, is also promising. Work by Bronstein et al. [110] on a physics-inspired continuous learning GNNs model that uses differential geometry and algebraic topology is a step towards overcoming MPNN paradigm limitations.

Graph Transformer Modeling: Transformers, a special case of GNNs [142], enhance graph data processing by incorporating topological information, offering superior capability in capturing complex node relationships and addressing missing elements and long-distance dependencies. This approach potentially reduces over-smoothing and over-squashing issues inherent in GNNs. The integration of Transformer architecture is anticipated to foster the development of Graph Transformers with enhanced expressive power, possibly outperforming traditional GNNs and expanding their application scope.

VI. CONCLUSION

GNNs research has seen substantial growth, yielding models with enhanced expressive power. Yet, a deeper grasp of this expressivity remains elusive. To advance understanding, we introduce a comprehensive theoretical framework that defines,

explores the limitations, and analyzes factors influencing GNNs expressive power. This framework also organizes current methods for boosting expressiveness. As it encompasses both graph features and topology, our approach provides a systematic methodology for studying GNNs expressive power, highlighting the need for methods that consider these dual aspects.

REFERENCES

- [1] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [2] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel, "Neural relational inference for interacting systems," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 2688–2697.
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [4] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua, "KGAT: Knowledge graph attention network for recommendation," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 950–958.
- [5] X. Yang, K. Tang, H. Zhang, and J. Cai, "Auto-encoding scene graphs for image captioning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 10685–10694.
- [6] Y. Chen, L. Wu, and M. J. Zaki, "Reinforcement learning based graph-to-sequence model for natural question generation," 2019, *arXiv:1908.04942*.
- [7] T. Ma, J. Chen, and C. Xiao, "Constrained generation of semantically valid graphs via regularizing variational autoencoders," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 7113–7124.
- [8] A. LeClair, S. Haque, L. Wu, and C. McMillan, "Improved code summarization via a graph neural network," in *Proc. 28th Int. Conf. Prog. Comprehension*, 2020, pp. 184–195.
- [9] Y. Tian, G.J. LiuWang, and M. Zhou, "ASA-GNN: Adaptive sampling and aggregation-based graph neural network for transaction fraud detection," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 3, pp. 3536–3549, Jun. 2024.
- [10] J. Zhou et al., "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.
- [11] L. Wang, X. Liu, X. Ma, J. Wu, J. Cheng, and M. Zhou, "A progressive quadric graph convolutional network for 3D human mesh recovery," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 1, pp. 104–117, Jan. 2023.
- [12] A. Markovitz, G. Sharir, I. Friedman, L. Zelnik-Manor, and S. Avidan, "Graph embedded pose clustering for anomaly detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 10539–10547.
- [13] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 249–270, Jan. 2022.
- [14] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [15] X. Liu, M. Yan, L. Deng, G. Li, X. Ye, and D. Fan, "Sampling methods for efficient training of graph convolutional networks: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 9, no. 2, pp. 205–234, Feb. 2022.
- [16] K. Wang, J. An, M. Zhou, Z. Shi, X. Shi, and Q. Kang, "Minority-weighted graph neural network for imbalanced node classification in social networks of internet of people," *IEEE Internet Things J.*, vol. 10, no. 1, pp. 330–340, Jan. 2023.
- [17] X. Wang, S. Zhao, L. Guo, L. Zhu, C. Cui, and L. Xu, "GraphCA: Learning from graph counterfactual augmentation for knowledge tracing," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 11, pp. 2108–2123, Nov. 2023.
- [18] S. Liu et al., "Inductive meta-path learning for schema-complex heterogeneous information networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10196–10209, Dec. 2024.
- [19] J. Zhu, X. Li, C. Gao, Z. Wang, and J. Kurths, "Unsupervised community detection in attributed networks based on mutual information maximization," *New J. Phys.*, vol. 23, no. 11, 2021, Art. no. 113016.
- [20] D. Hou, Z. Wang, C. Gao, and X. Li, "Sequential attention source identification based on feature representation," 2023, *arXiv:2306.15886*.
- [21] X. Cai, L. Xia, X. Ren, and C. Huang, "How expressive are graph neural networks in recommendation?," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 173–182.

- [22] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks," 2018, *arXiv:1806.01261*.
- [23] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Math. Control. Signals Syst.*, vol. 5, no. 4, pp. 455, 1992.
- [24] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Netw.*, vol. 4, no. 2, pp. 251–257, 1991.
- [25] H. Maron, E. N. F. Segol, and Y. Lipman, "On the universality of invariant networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 4363–4371.
- [26] W. Azizian and M. Lelarge, "Expressive power of invariant and equivariant graph neural networks," 2020, *arXiv:2006.15646*.
- [27] N. Keriven and G. Peyré, "Universal invariant and equivariant graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 7092–7101.
- [28] H. A. Helfgott, J. Bajpai, and D. Dona, "Graph isomorphisms in quasipolynomial time," 2017, *arXiv:1710.04574*.
- [29] J. Hartmanis, "Computers and intractability: A guide to the theory of NP-completeness (Michael R. Garey and David S. Johnson)," *SIAM Rev.*, vol. 24, no. 1, pp. 90–91, 1982.
- [30] E. Levin, R. Pieraccini, and W. Eckert, "A stochastic model of human-machine interaction for learning dialog strategies," *IEEE Trans. Speech Audio Process.*, vol. 8, no. 1, pp. 11–23, Jan. 2000.
- [31] H. Bunke, "Graph matching: Theoretical foundations, algorithms, and applications," in *Proc. Vis. Interface*, vol. 2000, pp. 82–88, 2000.
- [32] A. Zanfir and C. Sminchisescu, "Deep learning of graph matching," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2684–2693.
- [33] M. R. Garey and D. S. Johnson, "Computers and intractability. A guide to the theory of NP-completeness," *J. Symbolic Log.*, vol. 48, no. 2, pp. 498–500, 1983.
- [34] C. Morris et al., "Weisfeiler and leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4602–4609.
- [35] C. Morris, G. Rattan, and P. Mutzel, "Weisfeiler and Leman go sparse: Towards scalable higher-order graph embeddings," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21824–21840.
- [36] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," 2018, *arXiv:1810.00826*.
- [37] Z. Chen, S. Villar, L. Chen, and J. Bruna, "On the equivalence between graph isomorphism testing and function approximation with GNNs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 5868–15876.
- [38] R. Sato, "A survey on the expressive power of graph neural networks," 2020, *arXiv:2003.04078*.
- [39] M. Zhu, X. Wang, C. Shi, H. Ji, and P. Cui, "Interpreting and unifying graph neural networks with an optimization framework," in *Proc. Web Conf.*, 2021, pp. 1215–1226.
- [40] M. Balcilar, R. Guillaume, P. Héroux, B. Gaüzère, S. Adam, and P. Honeine, "Analyzing the expressive power of graph neural networks in a spectral perspective," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–26.
- [41] C. I. Kanatsoulis and A. Ribeiro, "Graph neural networks are more powerful than we think," 2022, *arXiv:2205.09801*.
- [42] M. DeRijke, "A note on graded modal logic," *Studia Logica*, vol. 64, no. 2, pp. 271–283, 2000.
- [43] J.-Y. Cai, M. Fürer, and N. Immerman, "An optimal lower bound on the number of variables for graph identification," *Combinatorica*, vol. 12, no. 4, pp. 389–410, 1992.
- [44] F. Geerts, J. Steegmans, and J. V. den Bussche, "On the expressive power of message-passing neural networks as global feature map transformers," in *Proc. Int. Symp. Found. Inf. Knowl. Syst.*, Springer, 2022, pp. 20–34.
- [45] M. Balcilar, P. Héroux, B. Gauzeur, P. Vasseur, S. Adam, and P. Honeine, "Breaking the limits of message passing graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 599–608.
- [46] M. Grohe, "The logic of graph neural networks," in *Proc. 36th Annu. ACM/IEEE Symp. Log. Comput. Sci.*, 2021, pp. 1–17.
- [47] P. Barceló et al., "The expressive power of graph neural networks as a query language," *ACM SIGMOD Rec.*, vol. 49, no. 2, pp. 6–17, 2020.
- [48] P. Barceló et al., "The logical expressiveness of graph neural networks," in *Proc. 8th Int. Conf. Learn. Representations*, 2020, pp. 1–21.
- [49] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 1–17.
- [50] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [51] K. Yoon et al., "Inference in probabilistic graphical models by graph neural networks," in *Proc. 53rd Asilomar Conf. Signals, Syst., Comput.*, 2019, pp. 868–875.
- [52] V. G. Satorras, Z. Akata, and M. Welling, "Combining generative and discriminative models for hybrid inference," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [53] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," *Adv. neural Inf. Process. Syst.*, 2017, pp. 13802–13812.
- [54] P. Battaglia et al., "Interaction networks for learning about objects, relations and physics," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4502–4510.
- [55] K. Xu, J. Li, M. Zhang, S. S. Du, K.-i. Kawarabayashi, and S. Jegelka, "What can neural networks reason about?," 2019, *arXiv:1905.13211*.
- [56] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, and X. Sun, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 3438–3445.
- [57] K. Oono and T. Suzuki, "Graph neural networks exponentially lose expressive power for node classification," 2019, *arXiv:1905.10947*.
- [58] M. Zhang, P. Li, Y. Xia, K. Wang, and L. Jin, "Revisiting graph neural networks for link prediction," 2020.
- [59] U. Alon and E. Yahav, "On the bottleneck of graph neural networks and its practical implications," 2020, *arXiv:2006.05205*.
- [60] J. Topping, F. D. Giovanni, B. P. Chamberlain, X. Dong, and M. M. Bronstein, "Understanding over-squashing and bottlenecks on graphs via curvature," 2021, *arXiv:2111.14522*.
- [61] J. You, J. M. Gomes-Selman, R. Ying, and J. Leskovec, "Identity-aware graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 10737–10745.
- [62] J. You, R. Ying, and J. Leskovec, "Position-aware graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 7134–7143.
- [63] V. Garg, S. Jegelka, and T. Jaakkola, "Generalization and representational limits of graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2020, pp. 3419–3430.
- [64] Z. Chen, L. Chen, S. Villar, and J. Bruna, "Can graph neural networks count substructures?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 10383–10395.
- [65] D. C. Elton, Z. Boukouvalas, M. D. Fuge, and P. W. Chung, "Deep learning for molecular design—a review of the state of the art," *Mol. Syst. Des. Eng.*, vol. 4, no. 4, pp. 828–849, 2019.
- [66] M. Sun, S. Zhao, C. Gilvary, O. Elemento, J. Zhou, and F. Wang, "Graph convolutional networks for computational drug development and discovery," *Brief. Bioinf.*, vol. 21, no. 3, pp. 919–935, 2020.
- [67] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Comput. Operations Res.*, vol. 134, 2021, Art. no. 105400.
- [68] N. Vesselinova, R. Steinert, F. D. Perez-Ramirez, and M. Boman, "Learning combinatorial optimization on graphs: A survey with applications to networking," *IEEE Access*, vol. 8, pp. 120388–120416, 2020.
- [69] H. Maron, H. Ben-Hamu, N. Shamir, and Y. Lipman, "Invariant and equivariant graph networks," 2018, *arXiv:1812.09902*.
- [70] F. Geerts and J. L. Reutter, "Expressiveness and approximation properties of graph neural networks," 2022, *arXiv:2204.04661*.
- [71] B. Weisfeiler and A. Leman, "The reduction of a graph to canonical form and the algebra which appears therein," *NTI, Ser.*, vol. 2, no. 9, pp. 12–16, 1968.
- [72] L. Babai and L. Kucera, "Canonical labelling of graphs in linear average time," in *Proc. 20th Annu. Symp. Found. Comput. Sci.*, 1979, pp. 39–46.
- [73] R. C. Read and D. G. Corneil, "The graph isomorphism disease," *J. Graph Theory*, vol. 1, no. 4, pp. 339–363, 1977.
- [74] N. Shervashidze, P. Schweitzer, E. J. V. Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-Lehman graph kernels," *J. Mach. Learn. Res.*, vol. 12, no. 9, pp. 2539–2561, 2011.
- [75] L. Babai, P. Erdos, and S. M. Selkow, "Random graph isomorphism," *SIAM J. Comput.*, vol. 9, no. 3, pp. 628–635, 1980.
- [76] B. Weisfeiler, *On Construction and Identification of Graphs*. vol. 558. Berlin, Germany: Springer, 2006.
- [77] M. Grohe and M. Otto, "Pebble games and linear equations," *J. Symbolic Log.*, vol. 80, no. 3, pp. 797–844, 2015.
- [78] L. Wu, P. Cui, J. Pei, L. Zhao, and X. Guo, "Graph neural networks: Foundation, frontiers and applications," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 4840–4841.

- [79] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Math. Control, Signals Syst.*, vol. 2, no. 4, pp. 303–314, 1989.
- [80] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 4438–4445.
- [81] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4805–4815.
- [82] S. Verma and Z.-L. Zhang, "Graph capsule convolutional neural networks," 2018, *arXiv:1805.08090*.
- [83] A. Santoro, F. Hill, D. Barrett, A. Morcos, and T. Lillicrap, "Measuring abstract reasoning in neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4477–4486.
- [84] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," 2018, *arXiv:1810.05997*.
- [85] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, "SIGN: Scalable inception graph neural networks," 2020, *arXiv:2004.11198*.
- [86] L. Chen, Z. Chen, and J. Bruna, "On graph neural networks versus graph-augmented MLPs," 2020, *arXiv:2010.15116*.
- [87] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [88] C. Yang, Q. Wu, J. Wang, and J. Yan, "Graph neural networks are inherently good generalizers: Insights by bridging GNNs and MLPs," 2022, *arXiv:2212.09034*.
- [89] A. Pearce, A. B. Wiltchko, B. Sanchez-Lengeling, and E. Reif, "A gentle introduction to graph neural networks," 2021. Accessed: Sep. 02, 2021. [Online]. Available: <https://staging.distill.pub/2021/gnn-intro/?ref=https://githubhelp.com>
- [90] O. Puny, D. Lim, B. Kiani, H. Maron, and Y. Lipman, "Equivariant polynomials for graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 28191–28222.
- [91] B. Zhang, J. Gai, Y. Du, Q. Ye, D. He, and L. Wang, "Beyond Weisfeiler-Lehman: A quantitative framework for GNN expressiveness," in *Proc. Twelfth Int. Conf. Learn. Representations*, 2024, pp. 1–73.
- [92] Z. Zhao, Z. Yang, C. Li, Q. Zeng, W. Guan, and M. Zhou, "Dual feature interaction-based graph convolutional network," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9019–9030, Sep. 2023.
- [93] A. Loukas, "What graph neural networks cannot learn: Depth vs width," 2019, *arXiv:1907.03199*.
- [94] C. Vignac, A. Loukas, and P. Frossard, "Building powerful and equivariant graph neural networks with structural message-passing," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 14143–14155.
- [95] C. Bodnar, F. Di Giovanni, B. P. Chamberlain, P. Liò, and M. M. Bronstein, "Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in GNNs," 2022, *arXiv:2202.04579*.
- [96] A. Loukas, "How hard is graph isomorphism for graph neural networks," 2020, *arXiv:2005.06649*.
- [97] P. de Haan, T. S. Cohen, and M. Welling, "Natural graph networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 3636–3646.
- [98] X. Wang, M. Zhu, D. Bo, P. Cui, C. Shi, and J. Pei, "AM-GCN: Adaptive multi-channel graph convolutional networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 1243–1253.
- [99] M. Hang, J. Neville, and B. Ribeiro, "A. collective learning framework to boost GNN expressiveness for node classification," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 4040–4050.
- [100] W. Bi, L. Du, Q. Fu, Y. Wang, S. Han, and D. Zhang, "MM-GNN: Mix-moment graph neural network towards modeling neighborhood feature distribution," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, 2023, pp. 132–140.
- [101] Z. Wang, Q. Cao, H. Shen, B. Xu, and X. Cheng, "Twin weisfeiler-lehman: High expressive GNNs for graph classification," 2022, *arXiv:2203.11683*.
- [102] G. Dasoulas, L. D. Santos, K. Scaman, and A. Virmaux, "Coloring graph neural networks for node disambiguation," 2019, *arXiv:1912.06058*.
- [103] R. Murphy, B. Srinivasan, V. Rao, and B. Ribeiro, "Relational pooling for graph representations," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 4663–4673.
- [104] R. Abboud, I. I. Ceylan, M. Grohe, and T. Lukasiewicz, "The surprising power of graph neural networks with random node initialization," 2020, *arXiv:2010.01179*.
- [105] P. Li, Y. Wang, H. Wang, and J. Leskovec, "Distance encoding: Design provably more powerful neural networks for graph representation learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4465–4478.
- [106] H. Wang, H. Yin, M. Zhang, and P. Li, "Equivariant and stable positional encoding for more powerful graph neural networks," 2022, *arXiv:2203.00199*.
- [107] B. Zhang, S. Luo, L. Wang, and D. He, "Rethinking the expressive power of GNNs via graph biconnectivity," 2023, *arXiv:2301.09505*.
- [108] Z. Zhang, P. Cui, J. Pei, X. Wang, and W. Zhu, "Eigen-GNN: A graph structure preserving plug-in for GNNs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2544–2555, Mar. 2023.
- [109] D. Zeng, W. Liu, W. Chen, L. Zhou, M. Zhang, and H. Qu, "Substructure aware graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 11129–11137.
- [110] G. Bouritsas, F. Frasca, S. P. Zafeiriou, and M. Bronstein, "Improving graph neural network expressivity via subgraph isomorphism counting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 657–668, Jan. 2023.
- [111] A. Wijesinghe and Q. Wang, "A new perspective on" how graph neural networks go beyond Weisfeiler-Lehman?," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–23.
- [112] L. Zhao, W. Jin, L. Akoglu, and N. Shah, "From stars to subgraphs: Uplifting any GNN with local structure awareness," 2021, *arXiv:2110.03753*.
- [113] M. Zhang and P. Li, "Nested graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 15734–15747.
- [114] C. Bodnar et al., "Weisfeiler and Lehman go topological: Message passing simplicial networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 1026–1037.
- [115] B. Bevilacqua et al., "Equivariant subgraph aggregation networks," 2021, *arXiv:2110.02910*.
- [116] G. Nikolentzos, G. Dasoulas, and M. Vazirgiannis, "K-hop graph neural networks," *Neural Netw.*, vol. 130, pp. 195–205, 2020.
- [117] J. Feng, Y. Chen, F. Li, A. Sarkar, and M. Zhang, "How powerful are k-hop message passing graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 4776–4790.
- [118] J. Feng et al., "Extending the design space of graph neural networks by rethinking folklore Weisfeiler-Lehman," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 9029–9064.
- [119] Q. Truong and P. Chin, "Weisfeiler and Lehman go paths: Learning topological features via path complexes," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 15382–15391.
- [120] N. Dehmamy, A.-L. Barabási, and R. Yu, "Understanding the representation power of graph neural networks in learning graph topology," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 15413–15423.
- [121] H. Pei, B. Wei, K. Chen-Chuan, Y. ChangLei, and B. Yang, "Geom-GCN: Geometric graph convolutional networks," 2020, *arXiv:2002.05287*.
- [122] Z. Huang, Y. Wang, C. Li, and H. He, "Going deeper into permutation-sensitive graph neural networks," 2022, *arXiv:2205.14368*.
- [123] H. Maron, H. Ben-Hamu, H. Serviansky, and Y. Lipman, "Provably powerful graph networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 2153–2164.
- [124] F. Geerts, "The expressive power of kth-order invariant graph networks," 2020, *arXiv:2007.12035*.
- [125] F. Frasca, B. Bevilacqua, M. M. Bronstein, and H. Maron, "Understanding and extending subgraph GNNs by rethinking their symmetries," 2022, *arXiv:2206.11140*.
- [126] R. Sato, M. Yamada, and H. Kashima, "Approximation ratios of graph neural networks for combinatorial problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 4081–4090.
- [127] R. Sato, M. Yamada, and H. Kashima, "Random features strengthen graph neural networks," in *Proc. SIAM Int. Conf. Data Mining*, SIAM, 2021, pp. 333–341.
- [128] V. Nanda, "Computational algebraic topology lecture notes," 2021. [Online]. Available: <https://people.maths.ox.ac.uk/nanda/cat/TDANotes.pdf>
- [129] A. Vikraman, F. Fuhlbrück, J. Köbler, and O. Verbitsky, "On weisfeiler-lehman invariance: Subgraph counts and related graph properties," *J. Comput. Syst. Sci.*, 2020, pp. 42–59.
- [130] S. Abu-El-Haija et al., "MixHop: Higher-order graph convolutional architectures via sparsified neighborhood mixing," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 21–29.
- [131] G. Wang, R. Ying, J. Huang, and J. Leskovec, "Multi-hop attention graph neural network," 2020, *arXiv:2009.14332*.
- [132] E. Chien, J. Peng, P. Li, and O. Milenkovic, "Adaptive universal generalized pagerank graph neural network," 2020, *arXiv:2006.07988*.

- [133] R. Brossard, O. Frigo, and D. Dehaene, "Graph convolutions that can finally model local structure," 2020, *arXiv:2011.15069*.
- [134] C. Ying et al., "Do transformers really perform badly for graph representation?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 28877–28888.
- [135] R. L. Murphy, B. Srinivasan, V. Rao, and B. Ribeiro, "Janossy pooling: Learning deep permutation-invariant functions for variable-size inputs," 2018, *arXiv:1811.01900*.
- [136] P. A. Papp, K. Martinkus, L. Faber, and R. Wattenhofer, "DropGNN: Random dropouts increase the expressiveness of graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 21997–22009.
- [137] P. A. Papp and R. Wattenhofer, "A theoretical comparison of graph neural network extensions," 2022, *arXiv:2201.12884*.
- [138] L. Cotta, C. Morris, and B. Ribeiro, "Reconstruction for powerful graph representations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 1713–1726.
- [139] C. Morris, K. Kersting, and P. Mutzel, "Glocalized weisfeiler-lehman graph kernels: Global-local feature maps of graphs," in *Proc. IEEE Int. Conf. Data Mining*, 2017, pp. 327–336.
- [140] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [141] M. D. Hawkins, "The problem of overfitting," *J. Chem. Inf. Comput. Sci.*, vol. 44, no. 1, pp. 1–12, 2004.
- [142] P. Veličković, "Everything is connected: Graph neural networks," *Curr. Opin. Struct. Biol.*, vol. 79, 2023, Art. no. 102538.



Kuihua Huang received the PhD degree from the National University of Defense Technology, China, in 2015. He is currently an associate professor with the National University of Defense Technology. His research interests include artificial intelligence, parallel simulation, and intelligent optimization.



Xiang Zhao received the PhD degree from the University of New South Wales, Australia, in 2014. He is currently a professor with the National University of Defense Technology, China. His research interests include graph data management and mining, with a special focus on knowledge graphs.



Bingxu Zhang received the BS degree in big data engineering from the National University of Defense Technology, Changsha, China, in 2021. She is currently working toward the PhD degree in management science and engineering with the School of Systems Engineering, National University of Defense Technology, Changsha, China. Her research interests include graph neural networks and combinatorial optimization.



Jincai Huang is currently a professor with the National University of Defense Technology, Changsha, Hunan, China; and a researcher with the Science and Technology on Information Systems Engineering Laboratory. His main research interests include artificial general intelligence, deep reinforcement learning, and multi-agent systems.



Changjun Fan received the BS, MS and PhD degrees all from the National University of Defense Technology, Changsha, China, in 2013, 2015, and 2020, respectively. He is also a visiting scholar with the Department of Computer Science, University of California, Los Angeles, for two years. He is currently an associate professor with the National University of Defense Technology. His research interests include deep graph learning and complex systems, with a special focus on their applications on intelligent decision making. During his previous study, he has published a

number of refereed journals and conference proceedings, such as nature machine intelligence, nature communications, AAAI, CIKM, etc.



Zhong Liu received the BS degree in physics from Central China Normal University, Wuhan, Hubei, China, in 1990, the MS degree in computer software and the PhD degree in management science and engineering both from the National University of Defense Technology, Changsha, China, in 1997 and 2000, respectively. He is a professor with the College of Systems Engineering, National University of Defense Technology, Changsha, China. His research interests include intelligent information systems, and intelligent decision making.



Shixuan Liu received the BS and PhD degrees from the National University of Defense Technology, Changsha, China, in 2019 and 2024, respectively. He is also a visiting scholar with the Department of Computer Science and Technology, Tsinghua University, where he has spent two years. He has published more than 10 papers in prestigious journals and conferences, including *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Cybernetics*, and *ICDM*, focusing on

knowledge reasoning and data mining.